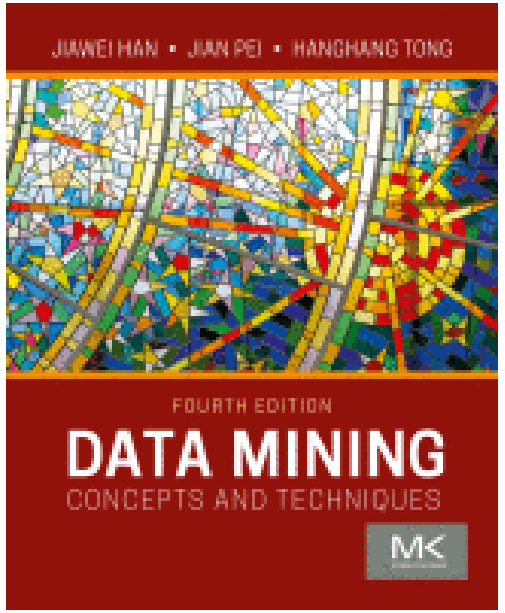
Deep Learning for Image Data

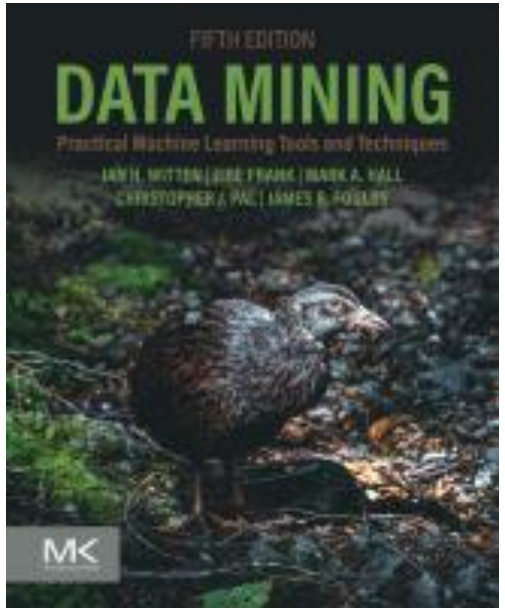
IF5141 dan IF4041 - Penambangan Data

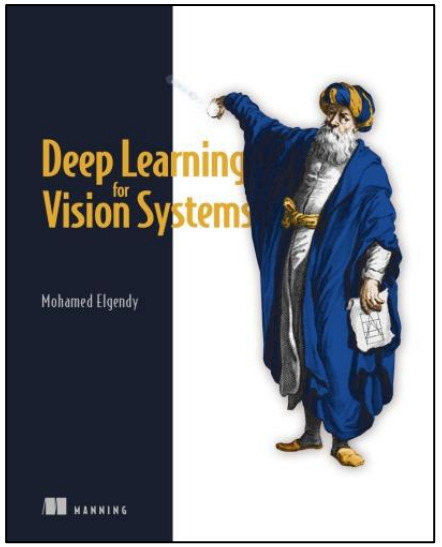
©2026 - Tim Pengajar IF4041 dan IF5141 Semester I TA 2026/2027

References

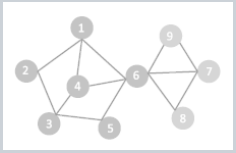
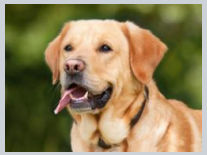
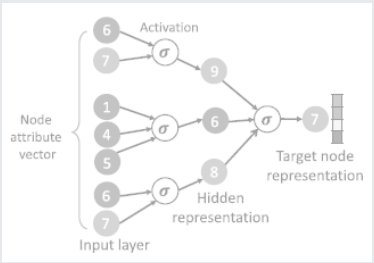
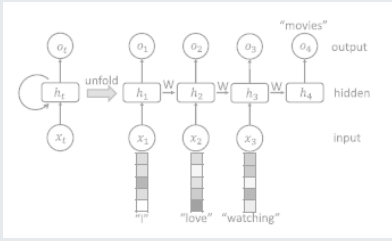
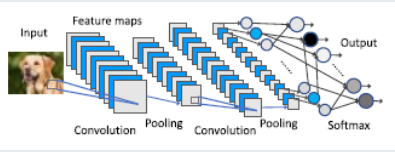
- Han, J., Pei, J., & Tong, H. (2024). *Data mining: Concepts and Techniques*. Morgan Kaufmann. https://doi.org/10.1016/C2013-0-18660-6
- Witten, I. H., Hall, M. A., Foulds, J. R., Frank, E., & Pal, C. J. (2025). *Data mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann. https://doi.org/10.1016/c2022-0-02092-4
- Elgendy, M. (2020). *Deep Learning for Vision Systems, First Edition*. Manning.







Overview of deep learning architecture

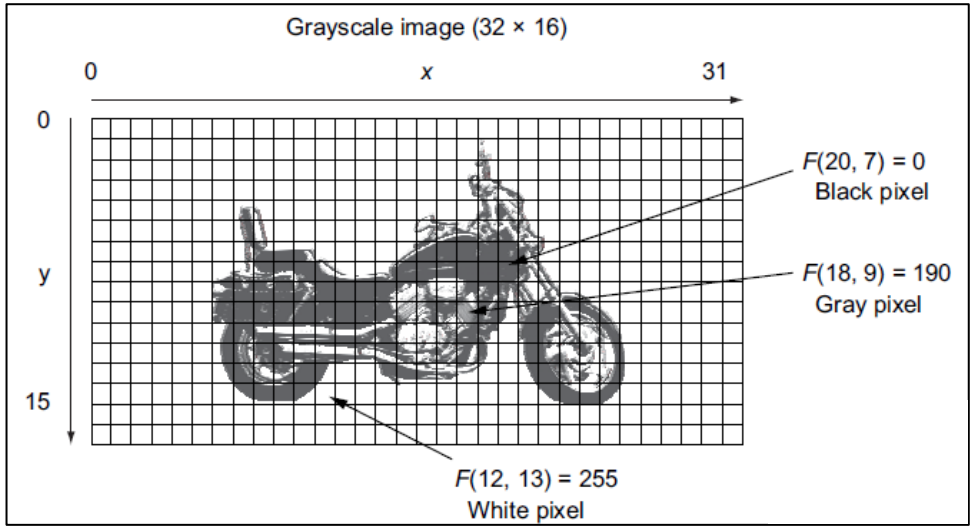
Data Type	*Multidimensional data* (i.e., tabular)	*Relational* data (i.e., graph)	*Sequence* data (i.e., text)	*Grid-like* data (i.e., image)
			x = "I love watching movies."	
Deep Learning (DL) Architecture	Feed-forward neural networks	Graph neural networks	Recurrent neural networks	Convolutional neural networks
				

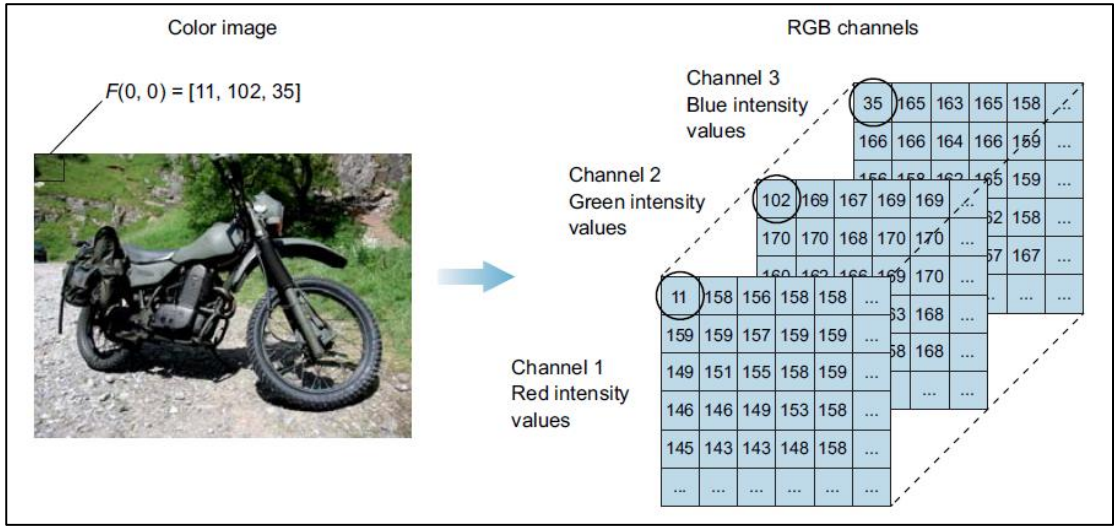
CNNs are a special kind of feedforward network that has proven extremely successful for image analysis.

Image: Grid-like Data

Image: Grid-like Data

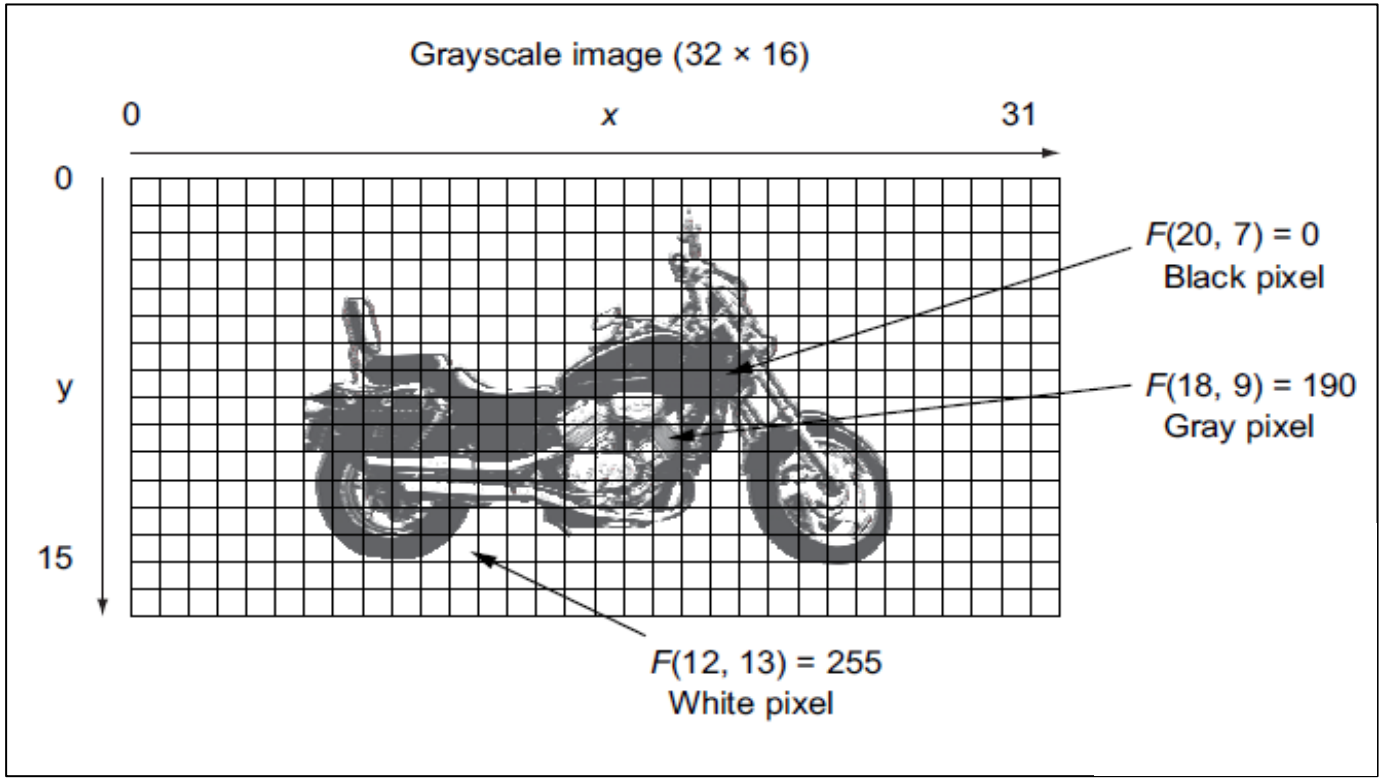
- The input grid data for CNNs are represented as a *multidimensional array* (i.e., a *tensor*).
- For example:
  - the gray image can be represented as a 2-D tensor (i.e., a matrix) whose elements measure the gray scales of the corresponding pixels; and
  - the color-image can be represented as a 3-D tensor with the third mode representing different color channels (e.g., red, green, blue).





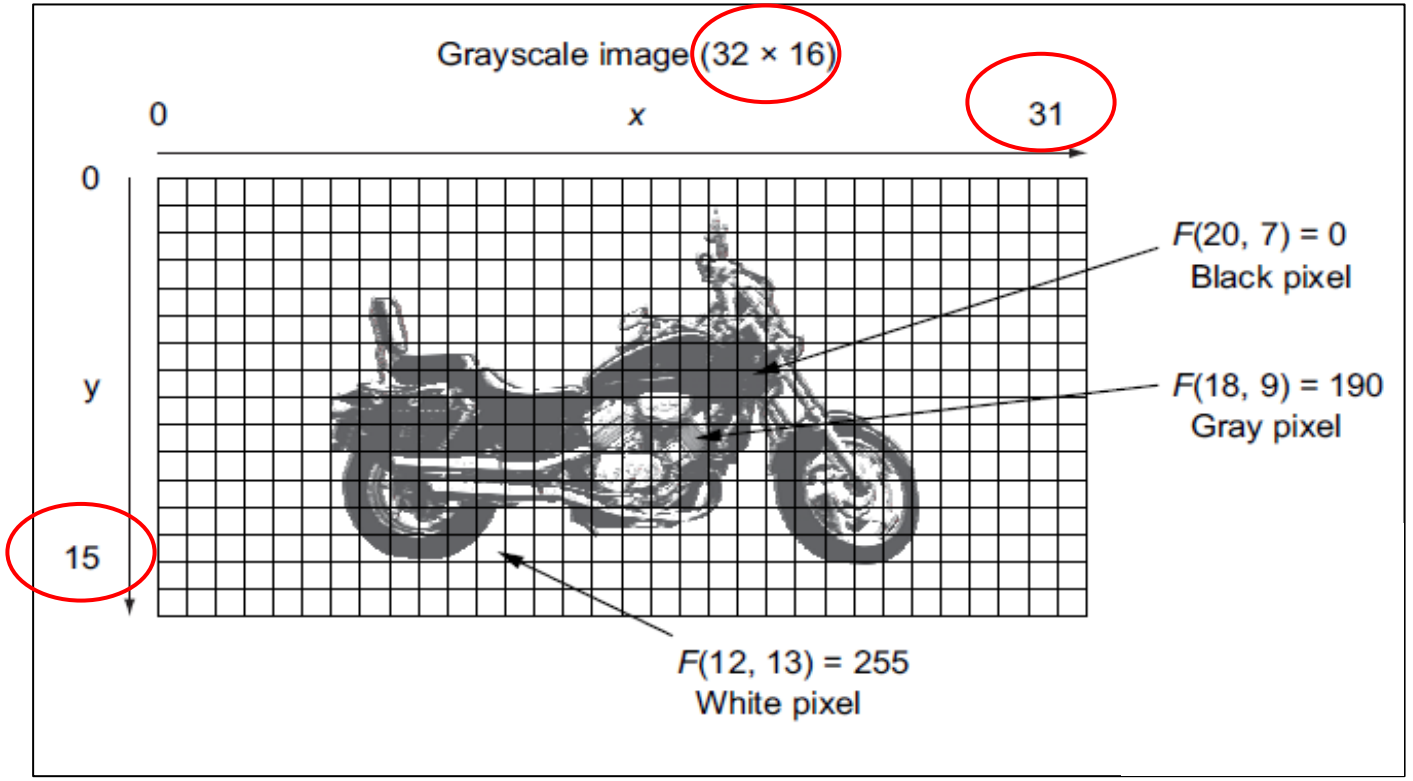
Pixel

- A digital image is made of a grid of pixels.
- The *pixel* is the raw building block of an image.
- The pixel values represent the ***intensity*** of light that appears in a given place in the image.



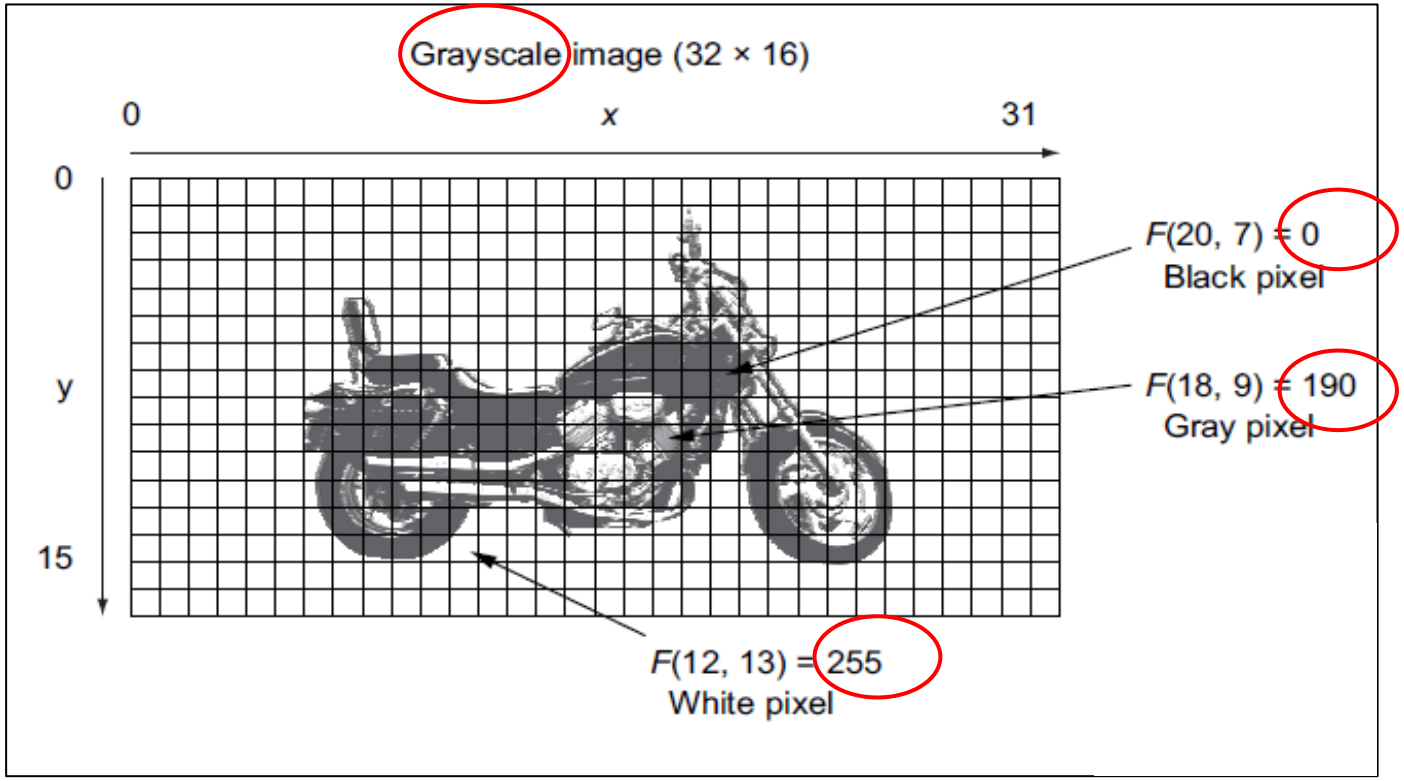
Pixel (2)

- This image has a size of 32 × 16.
  - This means the dimensions of the image are 32 pixels wide and 16 pixels tall.
- The x-axis goes from 0 to 31, and the y-axis from 0 to 15.
- Overall, the image has 512 (32 × 16) pixels.



Pixel (3)

- The pixel values range from 0 to 255.
- Since the pixel value represents the intensity of light,
  - the value 0 represents very dark pixels (black),
  - 255 is very bright (white),
  - and the values in between represent the intensity on the grayscale.



Pixel: How about color images?

- In color images, instead of representing the value of the pixel by just one number, the value is represented by three numbers representing the intensity of each color in the pixel.

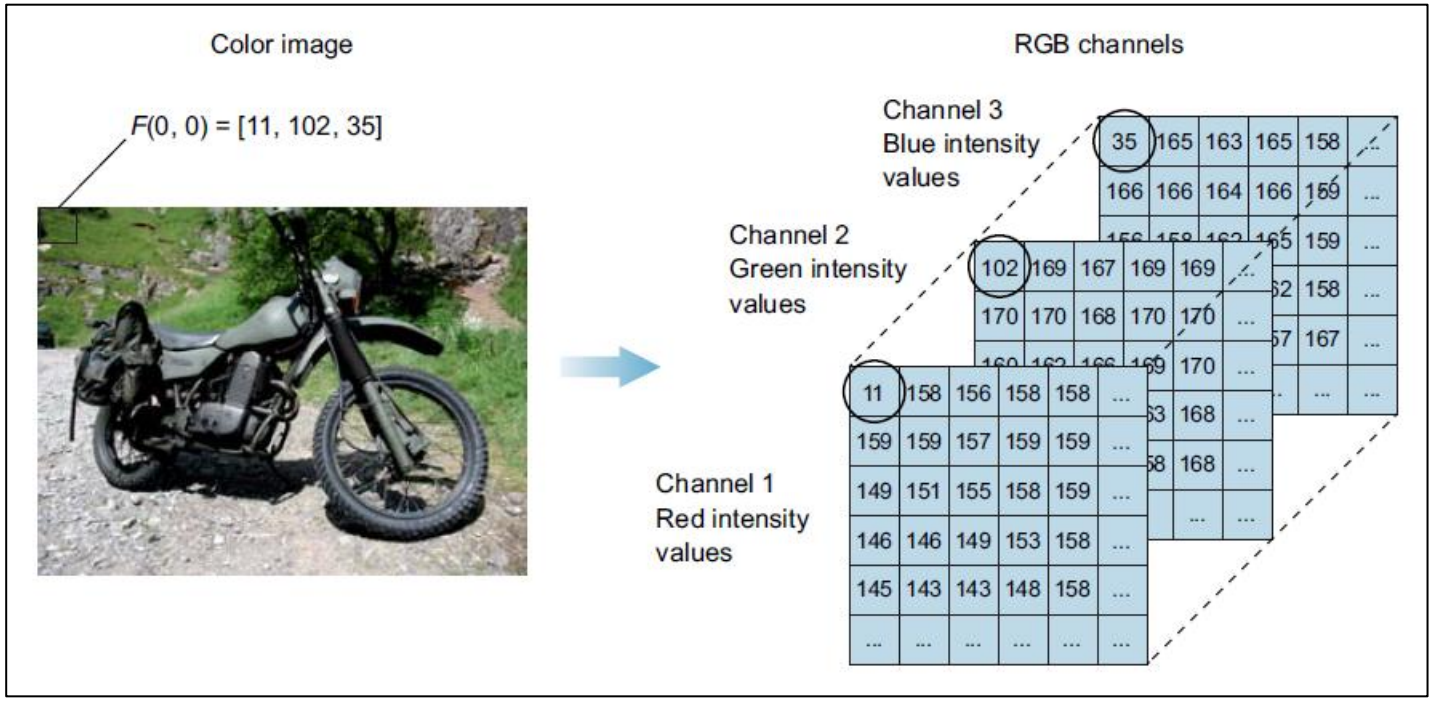
```
Grayscale => F(x, y) gives the intensity at position (x, y)
```

```
Color image in RGB => F(x, y) = [ red (x, y), green (x, y), blue (x, y) ]
```

  - In an RGB system, for example, the value of the pixel is represented by three numbers:
    - the intensity of red,
    - intensity of green, and
    - intensity of blue.

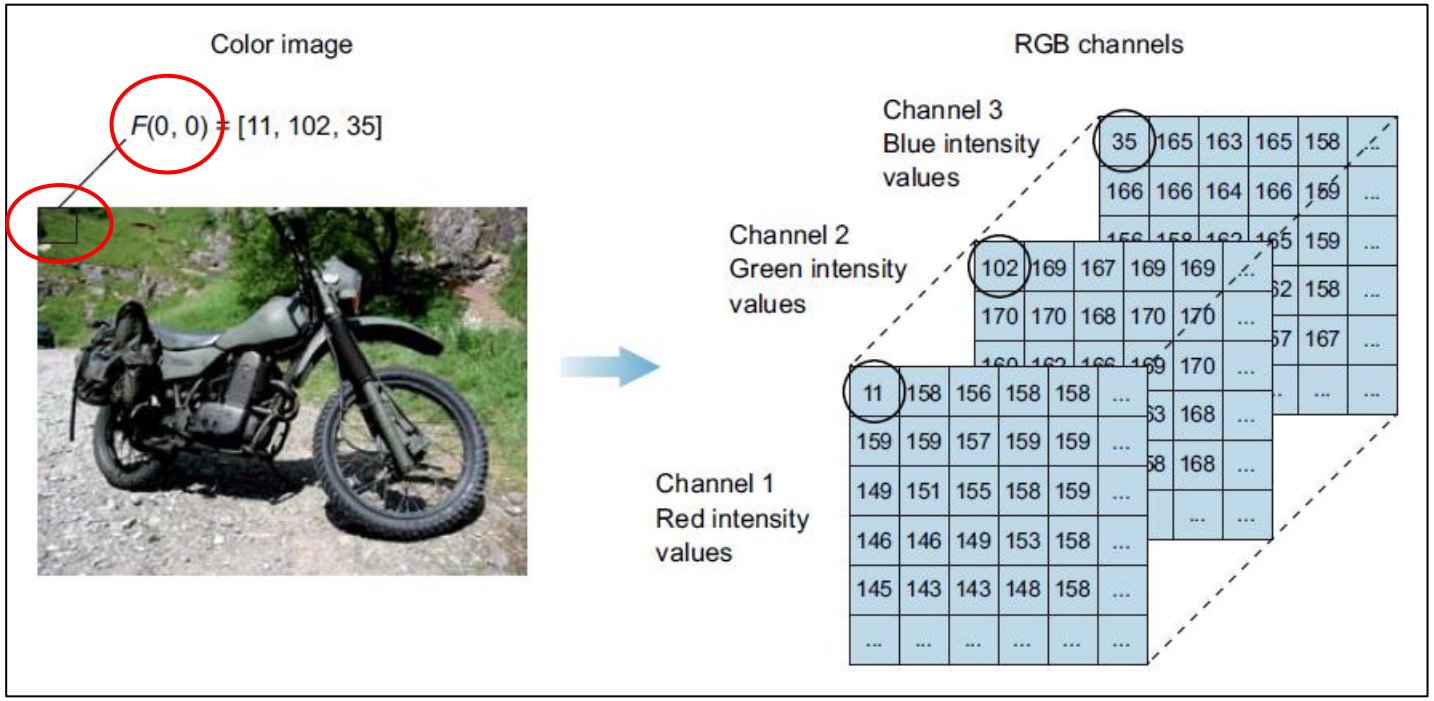
Pixel: How about color images? (2)

- The color image is composed of three channels: red, green, and blue.
  - We will have three matrices stacked on top of each other; that's why it's a 3D matrix.
- The dimensionality of 700 × 700 color images is (700, 700, 3).
  - This 700 × 700 size indicates the width and height of the image: there are 700 pixels horizontally and 700 vertically.



Pixel: How about color images? (3)

- If we take the pixel (0,0) as an example, we will see that it represents the top-left pixel of the **image of green grass.**
  - When we view this pixel in the color images, it looks like figure 1.18.



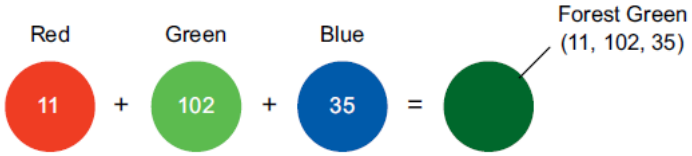


Figure 1.18 An image of green grass is actually made of three colors of varying intensity.

Pixel: How about color images? (4)

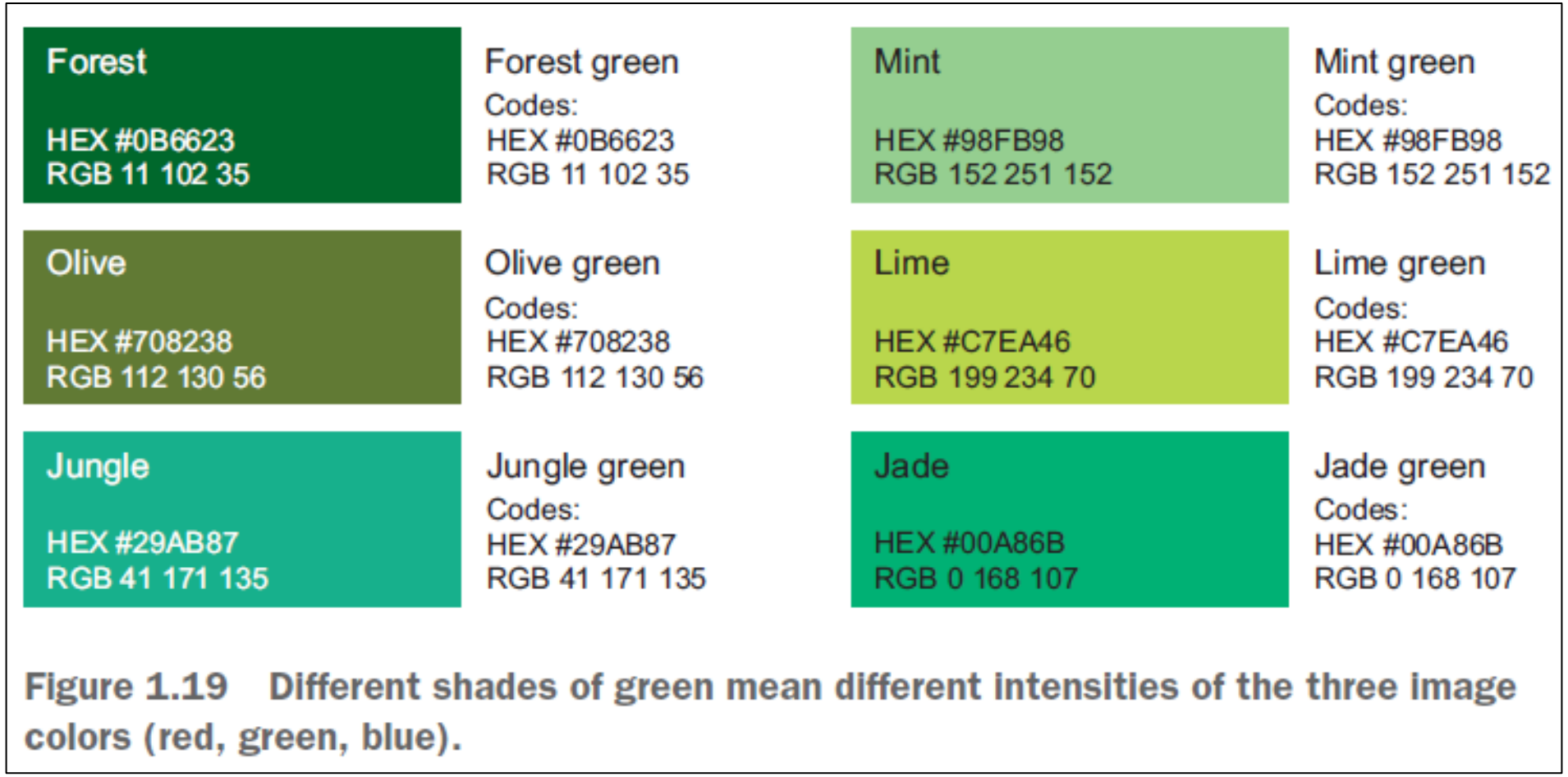
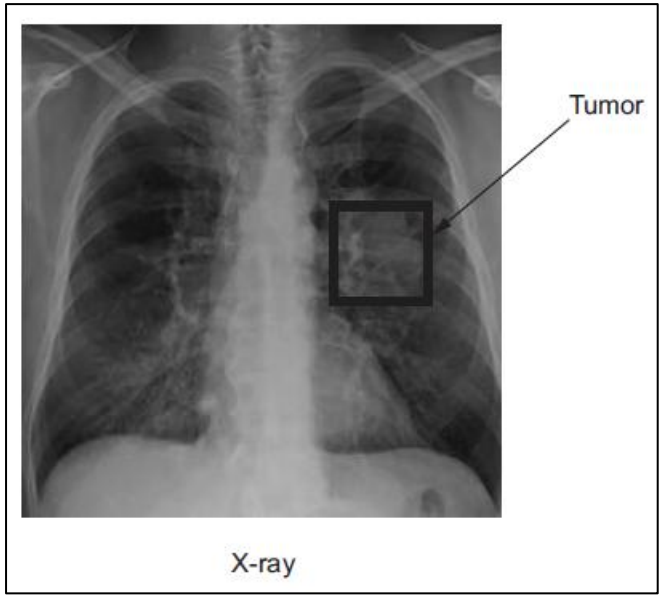


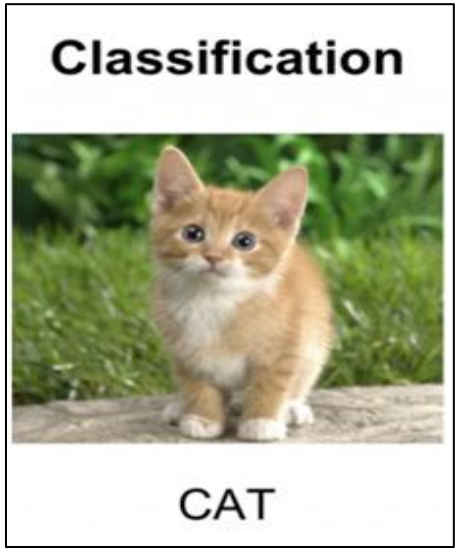
Figure 1.19 Different shades of green mean different intensities of the three image colors (red, green, blue).

Image Data Mining

Image Classification

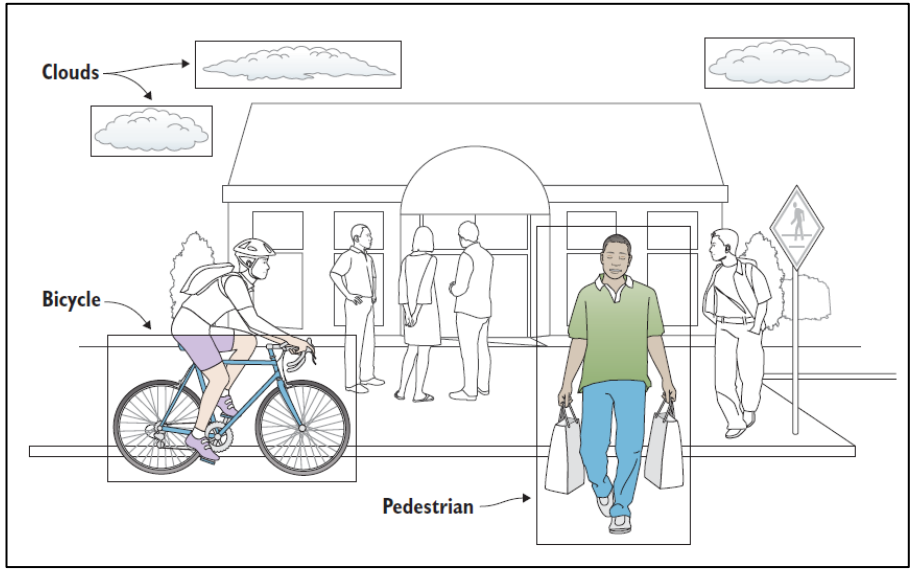
- *Image classification* is the task of assigning a label to an image from a predefined set of categories.
- If you have a set of *labeled* images, convolutional networks can classify them into a set of predefined classes.
- For example:
  - lung cancer diagnosis,
  - classifying different animals (like cats versus dogs versus horses),
  - different breeds of animals,
  - types of land suitable for agriculture, and so on.

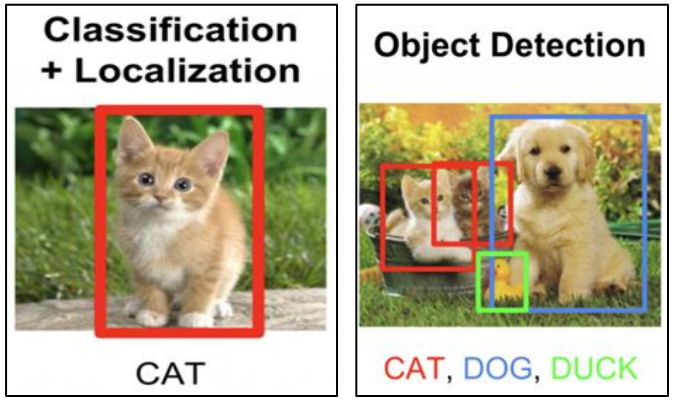




Object Detection and Localization

- This task is not only classify images but also can locate and detect each object in images that contain multiple objects.
- These DL systems can look at an image, break it up into smaller regions, and label each region with a class so that a variable number of objects in a given image can be localized and labeled.

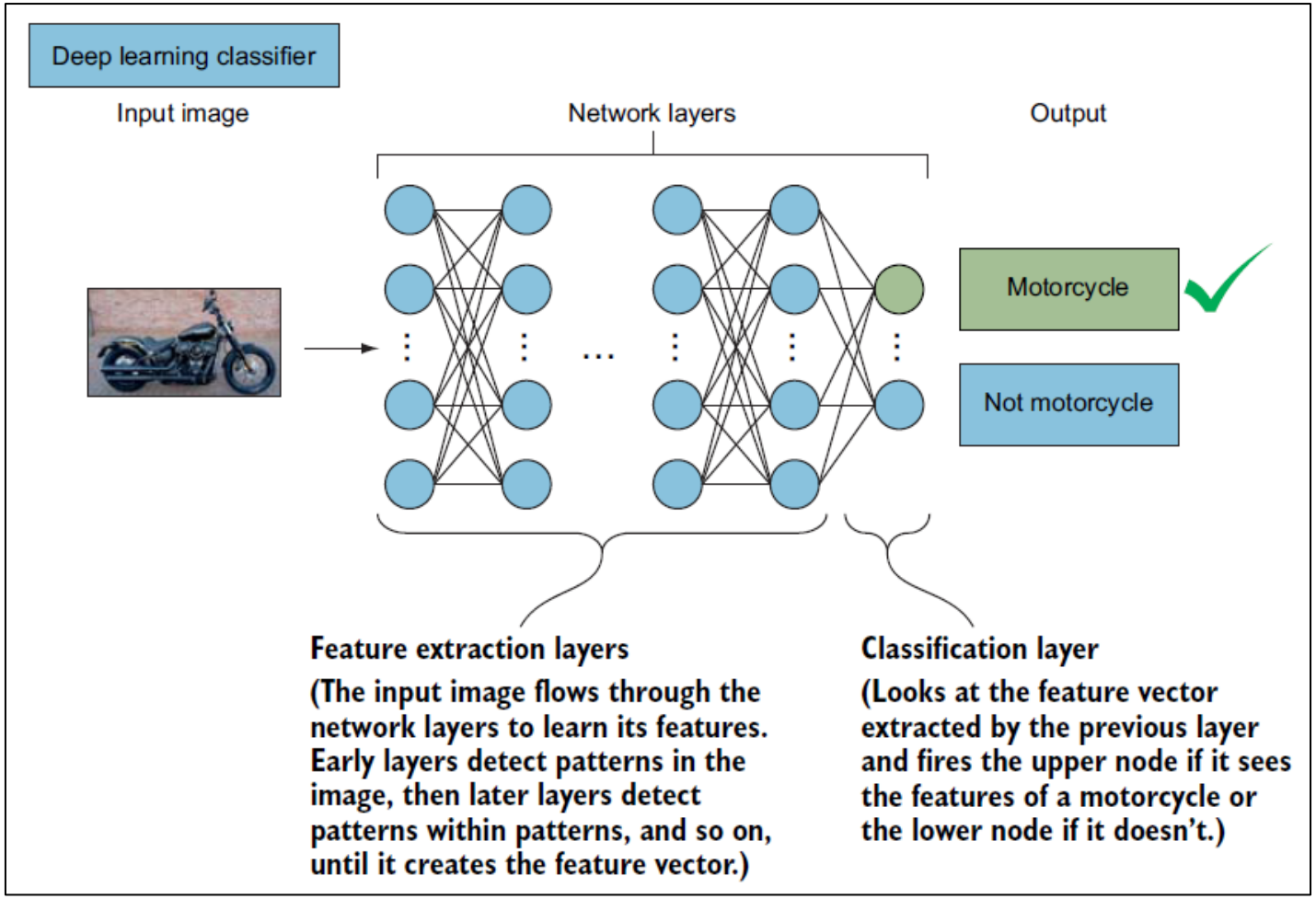




Preparing Image Data for DL

Input images pass through the layers of a neural network so it can learn features layer by layer.

The deeper your network is (the more layers), the more it will learn the features of the dataset



Elgendy (2020)

Data Preprocessing

- **Image Grayscaling**
  - Color images add computational complexity with their many parameters.
  - You can make a judgment call about converting all your images to grayscale, if your problem doesn't require color, to save on the computational complexity.
- **Image Resizing**
  - One limitation for neural networks (MLP, CNN, etc.) is that they require all images to be the same shape.
  - If we have three images with dimensions of 32 × 32, 28 × 28, and 64 × 64, we have to resize all the images to one size before feeding them to the model.

Data Preprocessing (2)

- **Data Normalization**
  - Data normalization is the process of rescaling your data to ensure that each input feature (pixel, in the image case) has a similar data distribution.
  - Often, raw images are composed of pixels with **varying scales** (ranges of values).
    - For example, one image may have a pixel value range from 0 to 255, and another may have a range of 20 to 200.
  - Although not required, it is preferred to normalize the pixel values to the range of 0 to 1 to boost learning performance and make the network converge **faster**.

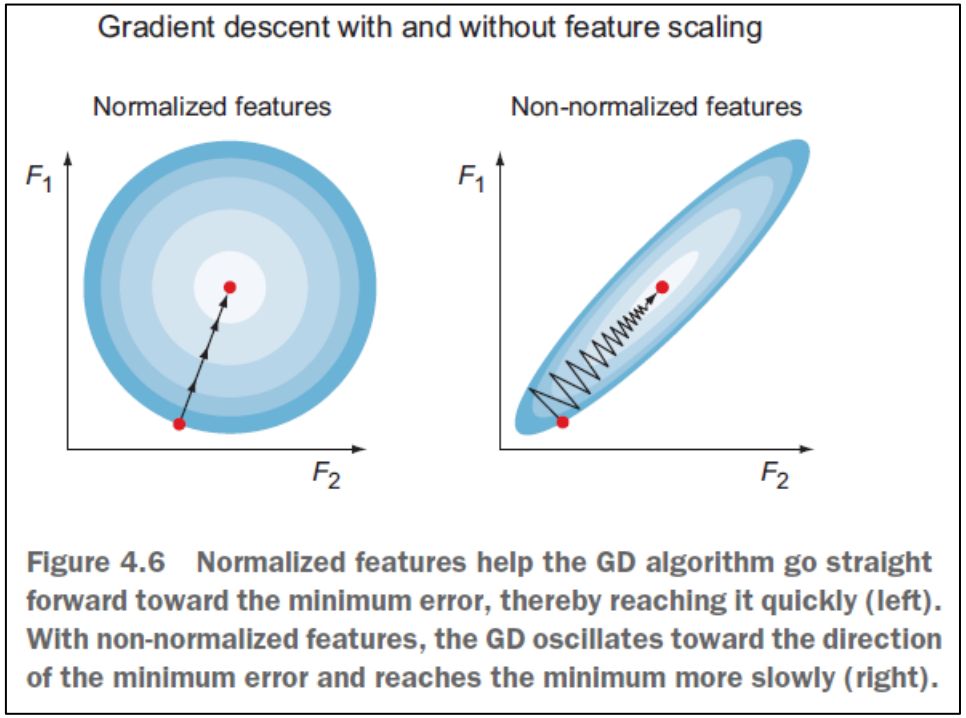


Figure 4.6 Normalized features help the GD algorithm go straight forward toward the minimum error, thereby reaching it quickly (left). With non-normalized features, the GD oscillates toward the direction of the minimum error and reaches the minimum more slowly (right).

DL Architecture

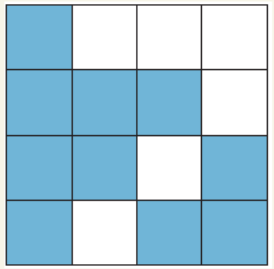
CNN architectures

Problems in MLPs that CNNs are designed to fix

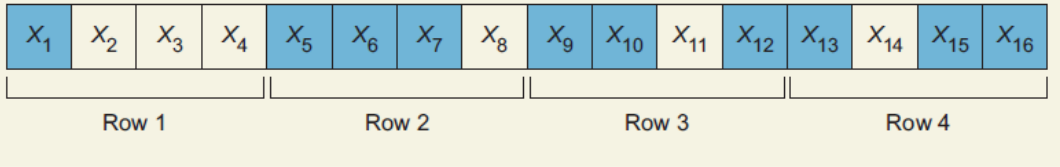
- Before feeding an image to the hidden layers of an MLP, we must flatten the image matrix to a 1D vector.
  - MLPs have no knowledge of the fact that these pixel numbers were originally spatially arranged in a grid and that they are connected to each other.
- CNNs, do not require a flattened image.
  - We can feed the raw image matrix of pixels to a CNN network, and the CNN will understand that pixels that are close to each other are more heavily **related** than pixels that are far apart.

Visualizing input vectors

To help visualize the flattened input vector, let's look at a much smaller matrix (4, 4):



The input (x) is a flattened vector with the dimensions (1, 16):



So, if we have pixel values of 0 for black and 255 for white, the input vector will be as follows:

Input = [0, 255, 255, 255, 0, 0, 0, 255, 0, 0, 255, 0, 0, 255, 0, 0]

Problems in MLPs that CNNs are designed to fix (2)

- MLPs are composed of dense layers that are fully connected to each other.
  - *Fully connected* means every node in one layer is connected to *all* nodes of the previous layer and *all* nodes in the next layer.
  - If we have an image with dimensions 1,000 × 1,000, it will yield 1 million parameters for each node in the first hidden layer.
    - So if the first hidden layer has 1,000 neurons, this will yield 1 billion parameters even in such a small network.
    - You can imagine the computational complexity of optimizing 1 billion parameters **after only the first layer**.
- CNNs are *locally connected* layers, nodes are connected to only a small subset of the previous layers' nodes.
  - Locally connected layers use far fewer parameters than densely connected layers.

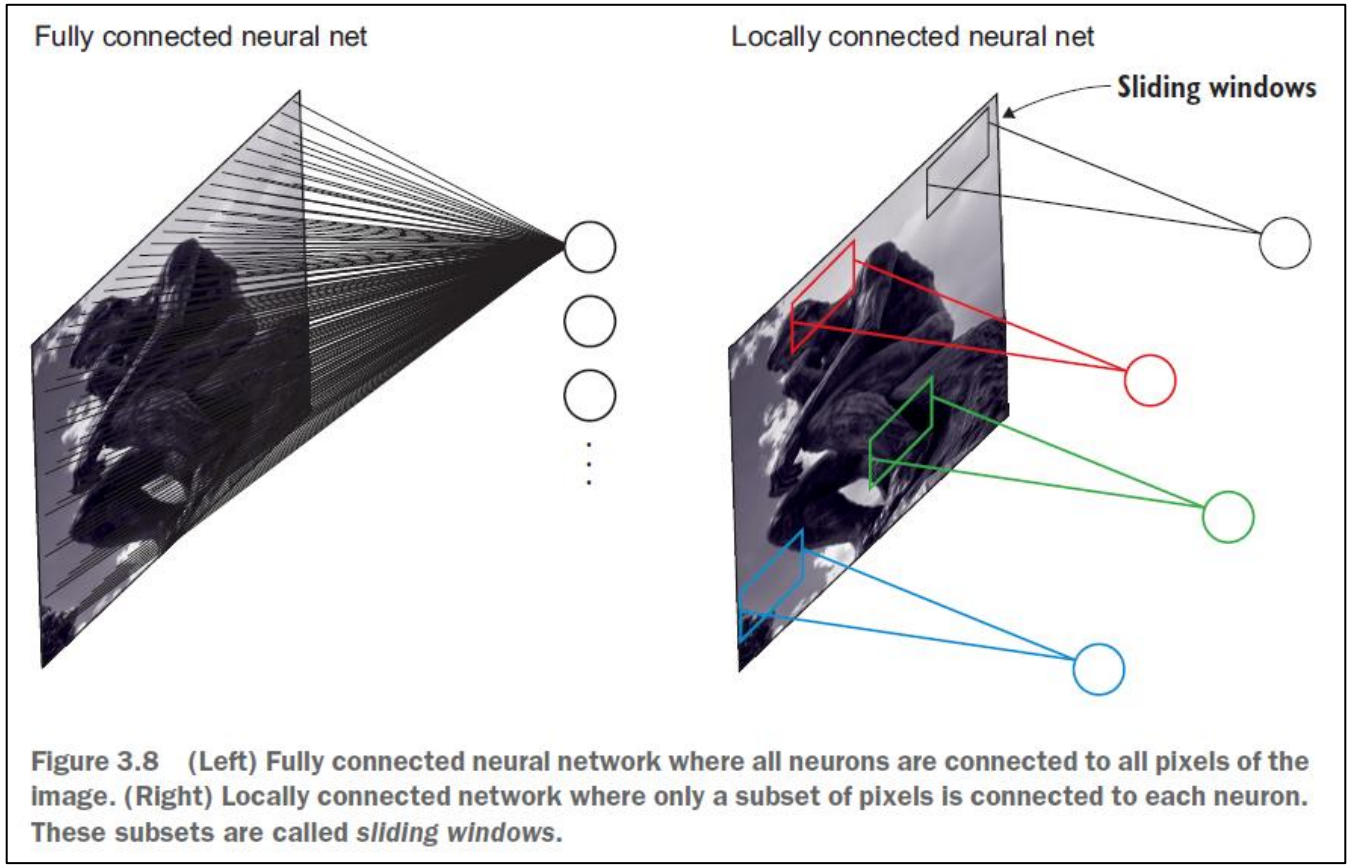
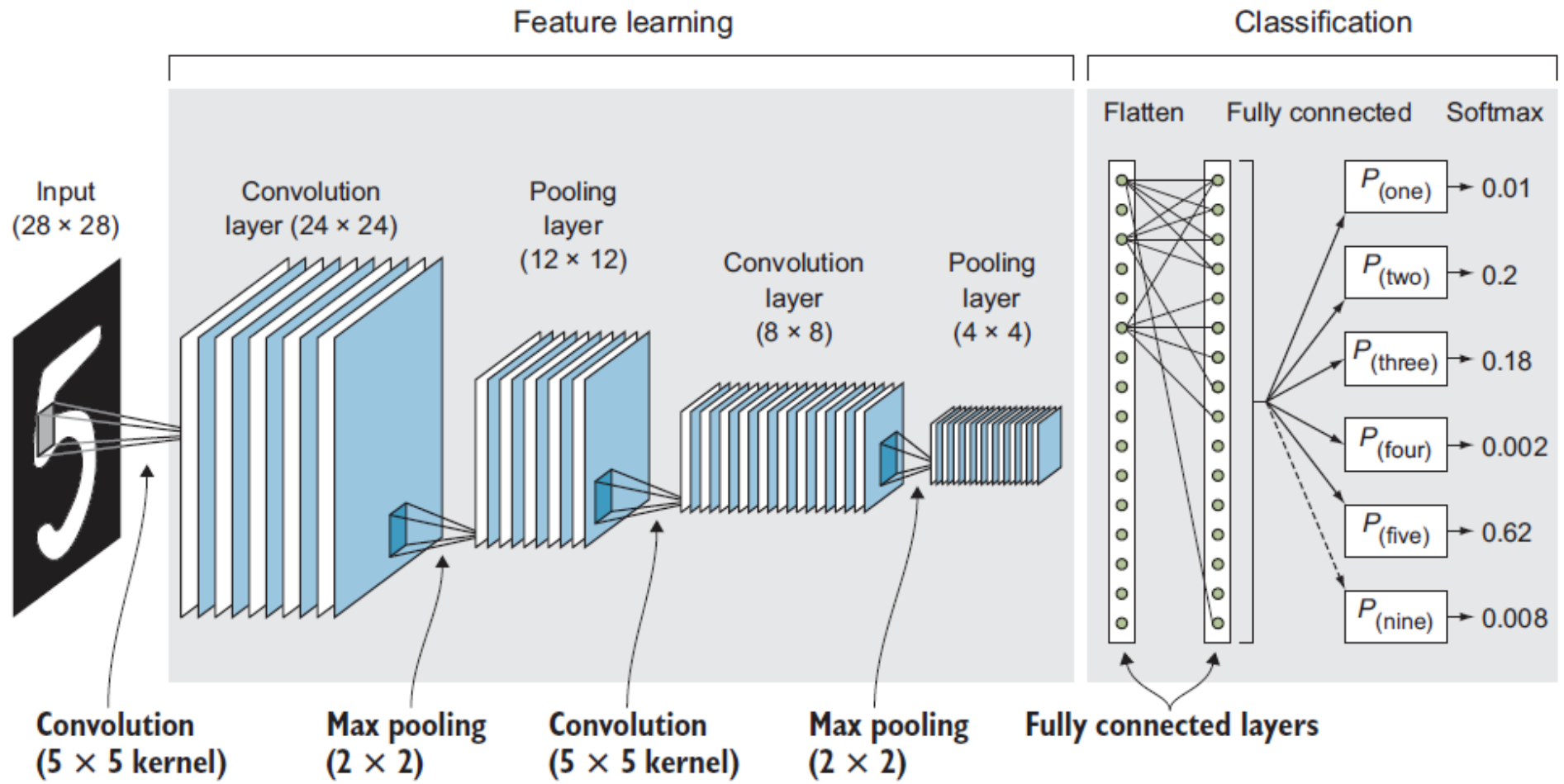


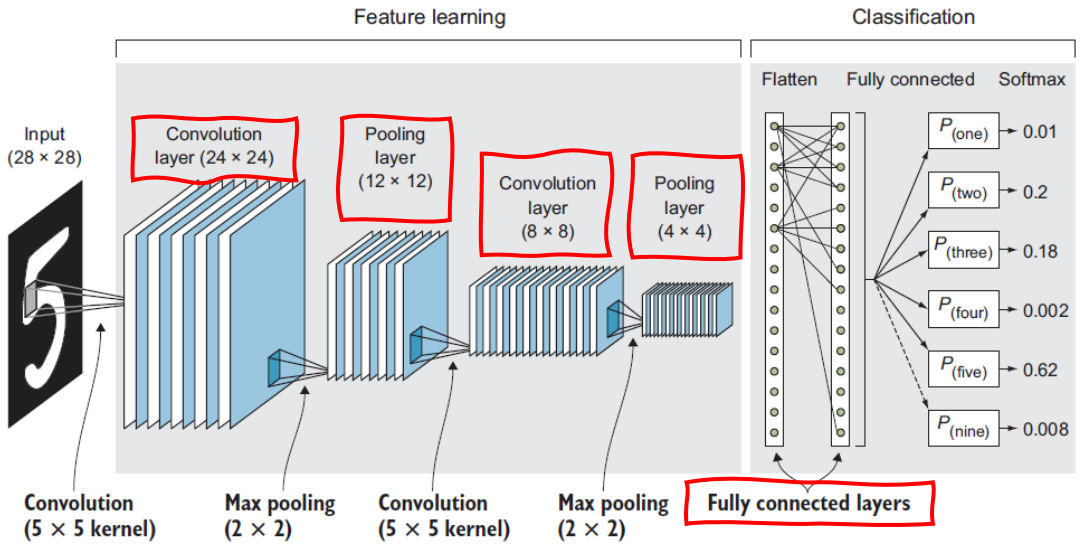
Figure 3.8 (Left) Fully connected neural network where all neurons are connected to all pixels of the image. (Right) Locally connected network where only a subset of pixels is connected to each neuron. These subsets are called *sliding windows*.

CNN Architecture



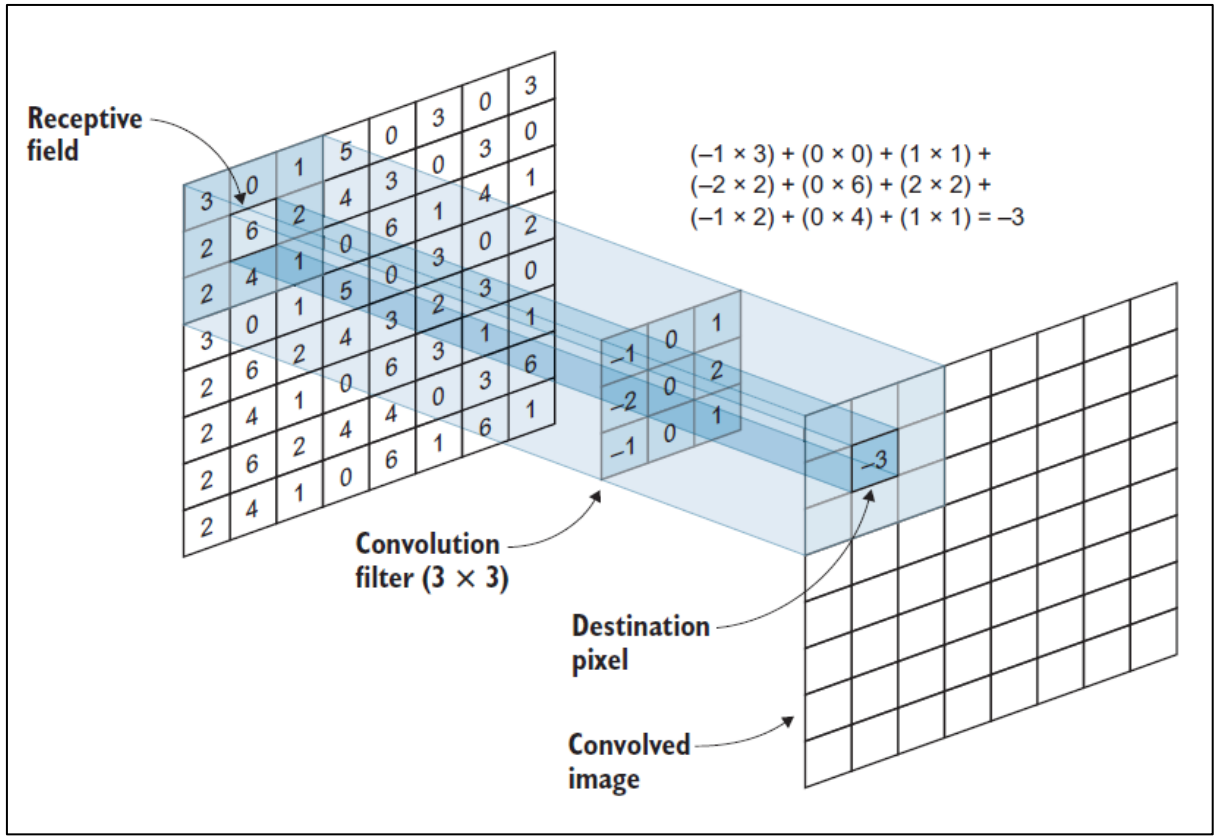
Components in CNN architecture

- There are three main types of layers in CNN:
  1. Convolutional layer (CONV)
  2. Pooling layer (POOL)
  3. Fully connected layer (FC)
- The convolutional layers are for feature learning or extraction.
- The fully connected layers are for classification.
- You can add as many convolutional and fully connected layers as you see fit.



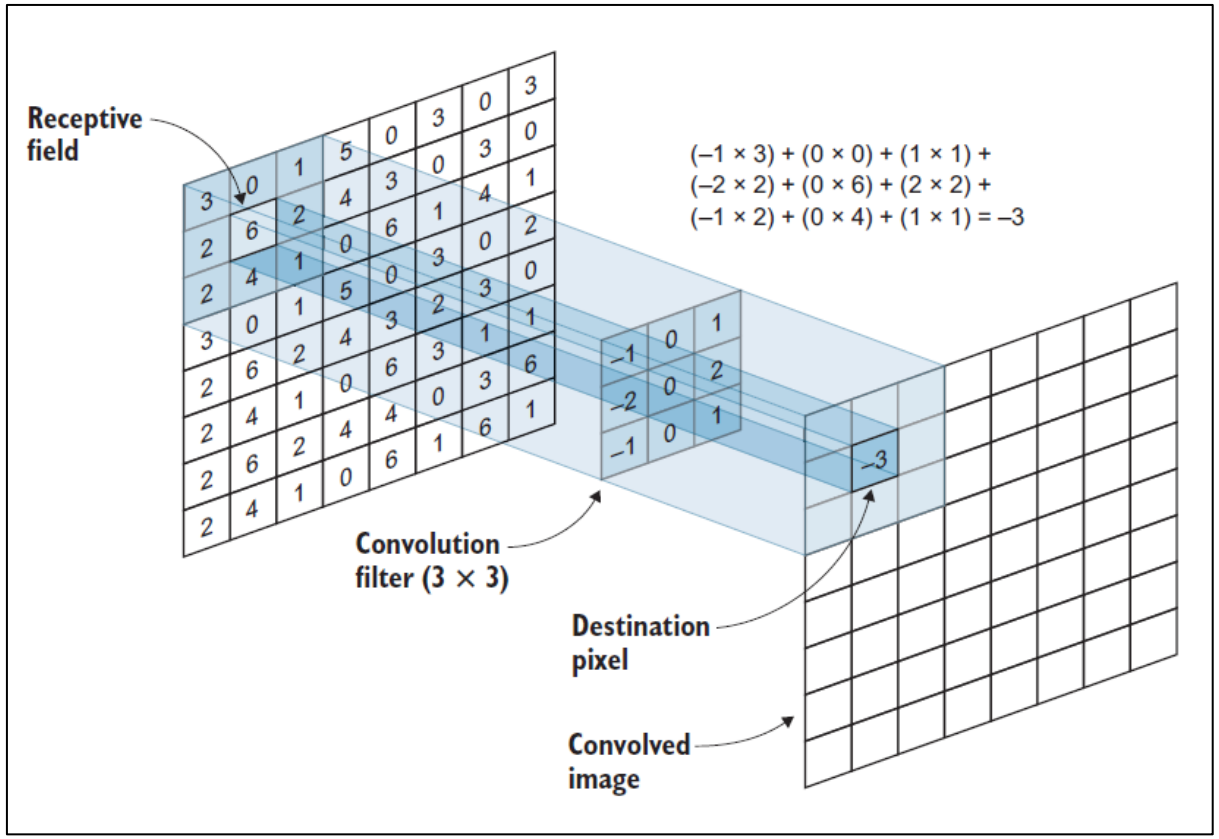
Convolutional Layer

- In mathematics, convolution is the **operation** of **two functions** to **produce** a **third modified function**.
- In the context of CNNs:
  - the first function is the **input image**, and
  - the second function is the **convolutional filter**.
  - We will perform some mathematical operations to **produce a modified image** with new pixel values.



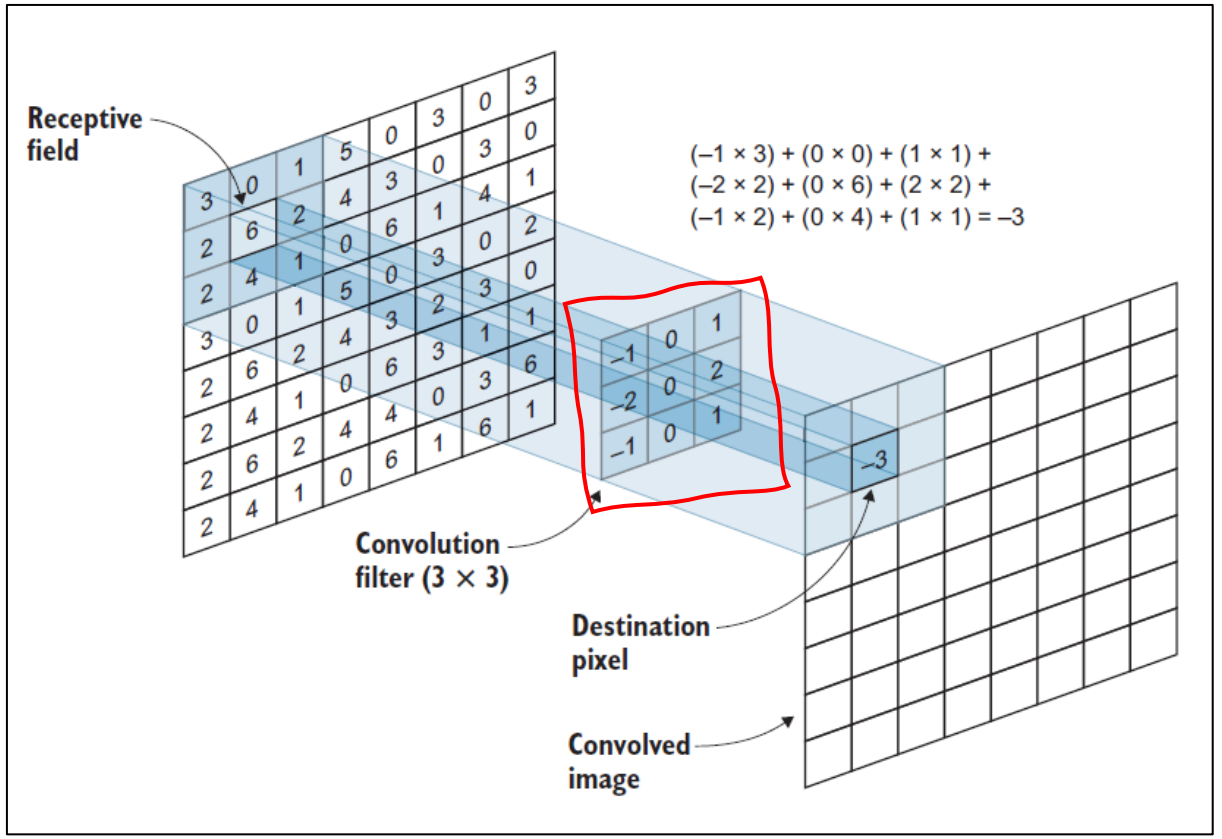
Convolutional Layer (2)

- Convolutional layers act like a feature finder window that **slides** over the image pixel by pixel to extract meaningful features that identify the objects in the image.
- By sliding the convolutional filter over the input image, the network **breaks** the image into little chunks and **processes** those chunks individually to **assemble the modified image**, a feature map.



Convolution Filter (or Kernel)

- The small 3 × 3 matrix in the middle *is* the convolution filter, also called a *kernel*.
  - The kernel slides over the original image pixel by pixel and does some math calculations to get the values of the new "convolved" image on the next layer.
    - This convolved image is called a *feature map* or *activation map*.
- What are the kernel values?
  - In CNNs, the convolution matrix *is* the weights.
    - This means they are also *randomly initialized* and the values are *learned* by the network.



Convolution Filter (Kernel) (2)

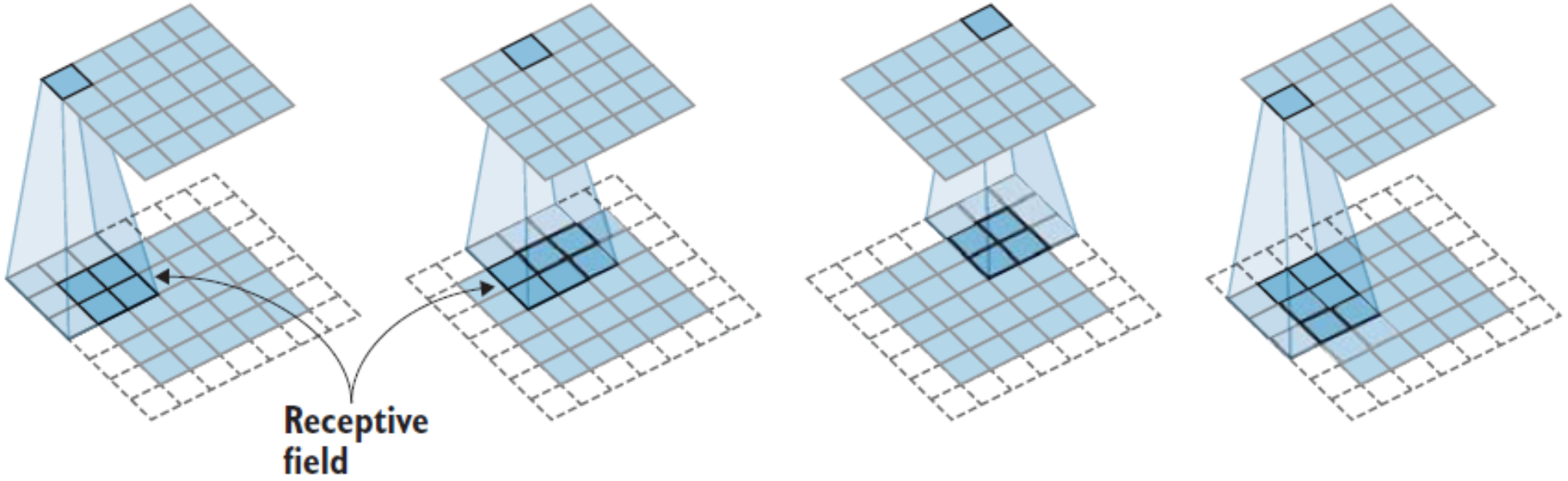
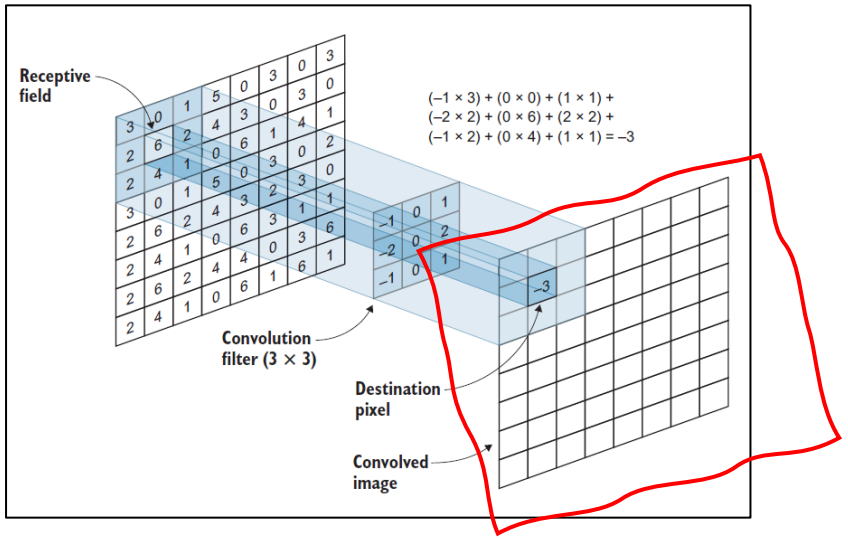
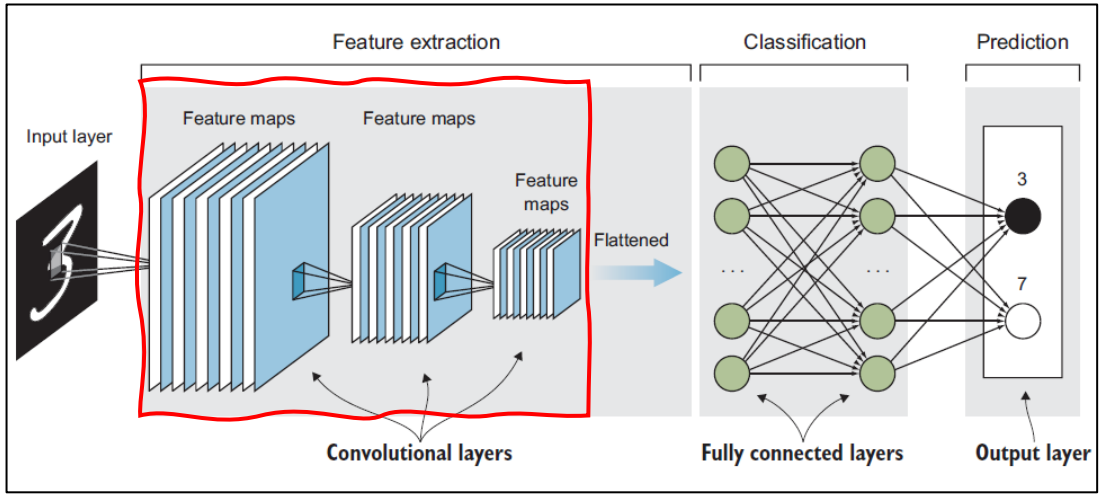


Figure 3.14 The kernel slides over the original image pixel by pixel and calculates the convolved image on the next layer. The convolved area is called the *receptive field*.

Feature Maps

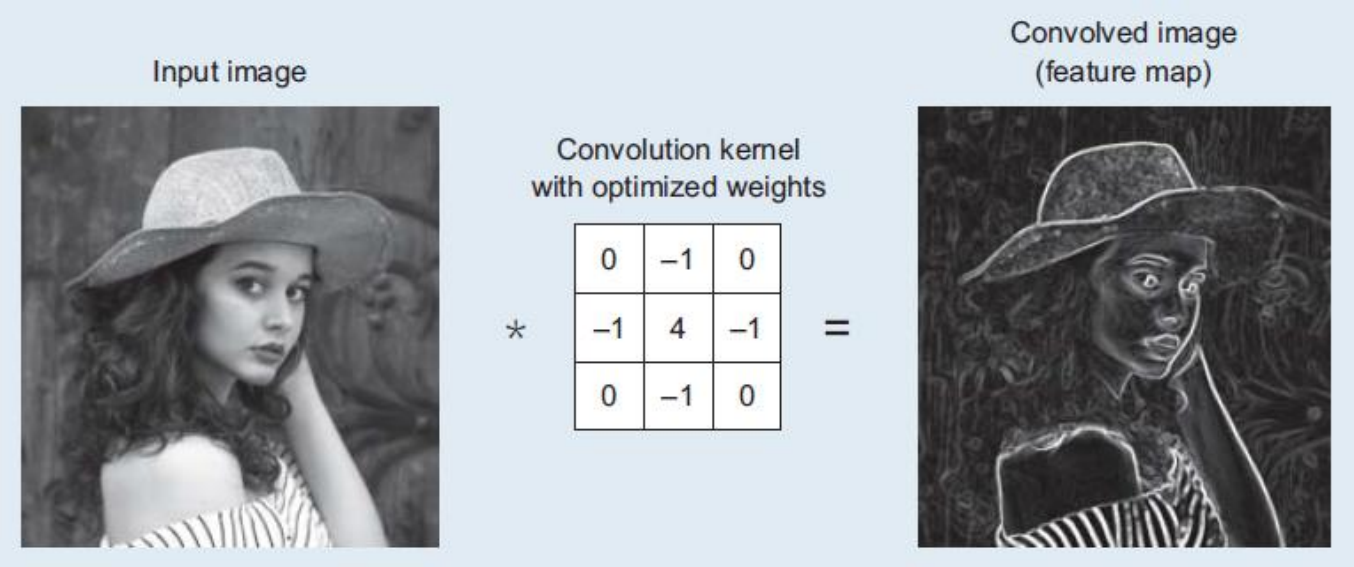
- The basic idea of neural networks is that neurons learn features from the input.
  - CNNs look for features such as straight lines, edges, or even objects. Whenever they spot these features, they report them to the **feature map**.
- In CNNs, a *feature map* is the output of one filter applied to the previous layer.
  - It is called a feature map because it is a mapping of where a certain kind of feature is found in the image.
  - Each feature map is looking for something specific: one could be looking for straight lines and another for curves.





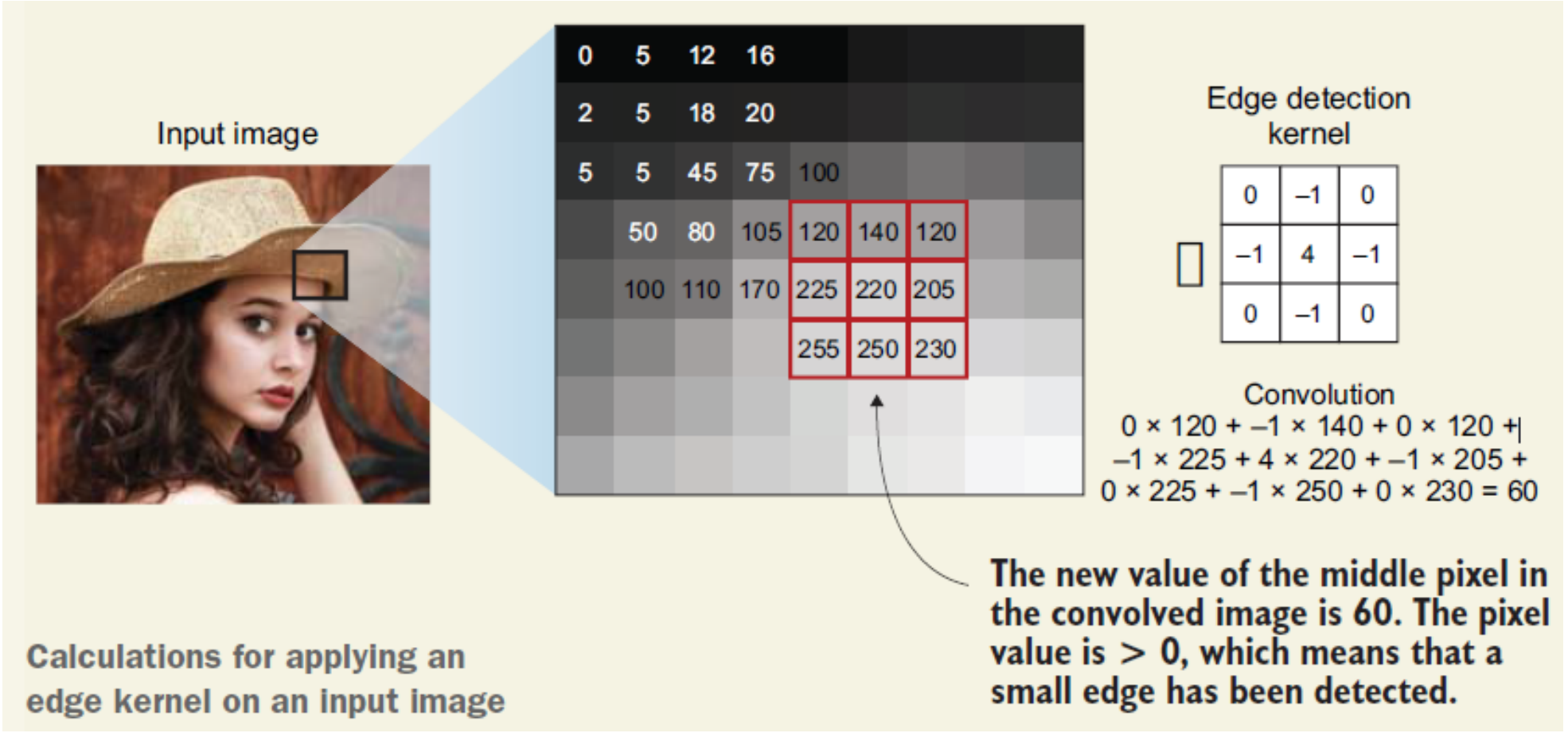
Why applying filters?

- Filters are used to filter out unwanted information or **amplify features** in an image.
  - Look at this **edge-detection filter**:



When this kernel (K) is convolved with the input image F(x,y), it creates a new convolved image (a feature map) that **amplifies the edges**.

How the convolution happens?

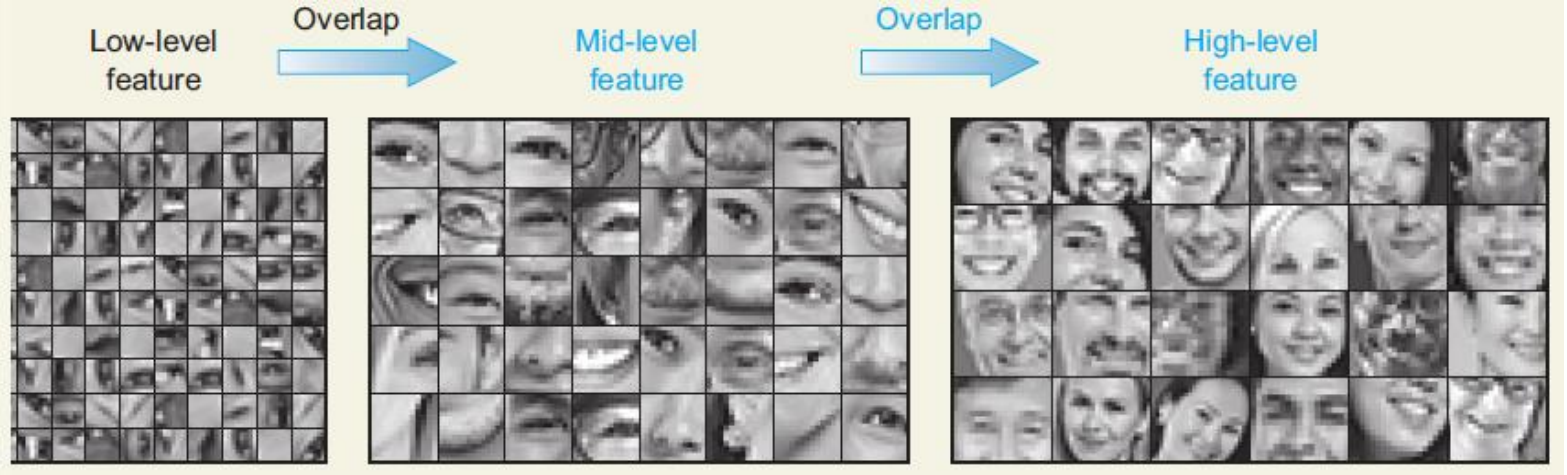


Calculations for applying an edge kernel on an input image

Elgendy (2020)

Other filters can be applied to detect different types of features

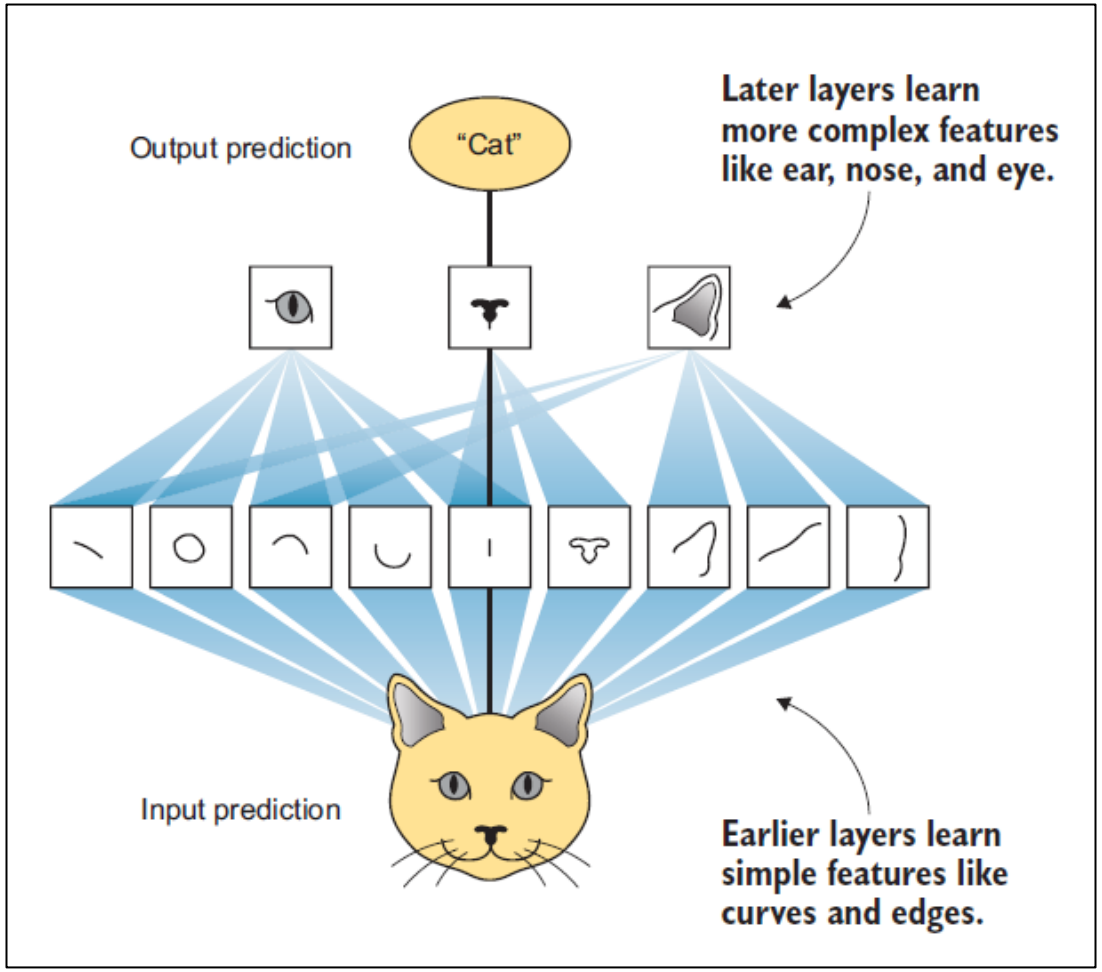
- For example, some filters detect horizontal edges, others detect vertical edges, still others detect more complex shapes like corners, and so on.
- The point is that these filters, when applied in the convolutional layers, yield the feature-learning behavior:
  - First, they learn simple features like **edges and straight lines**, and later layers learn more complex features.



Elgendy (2020)

how the network learns features throughout its layers?

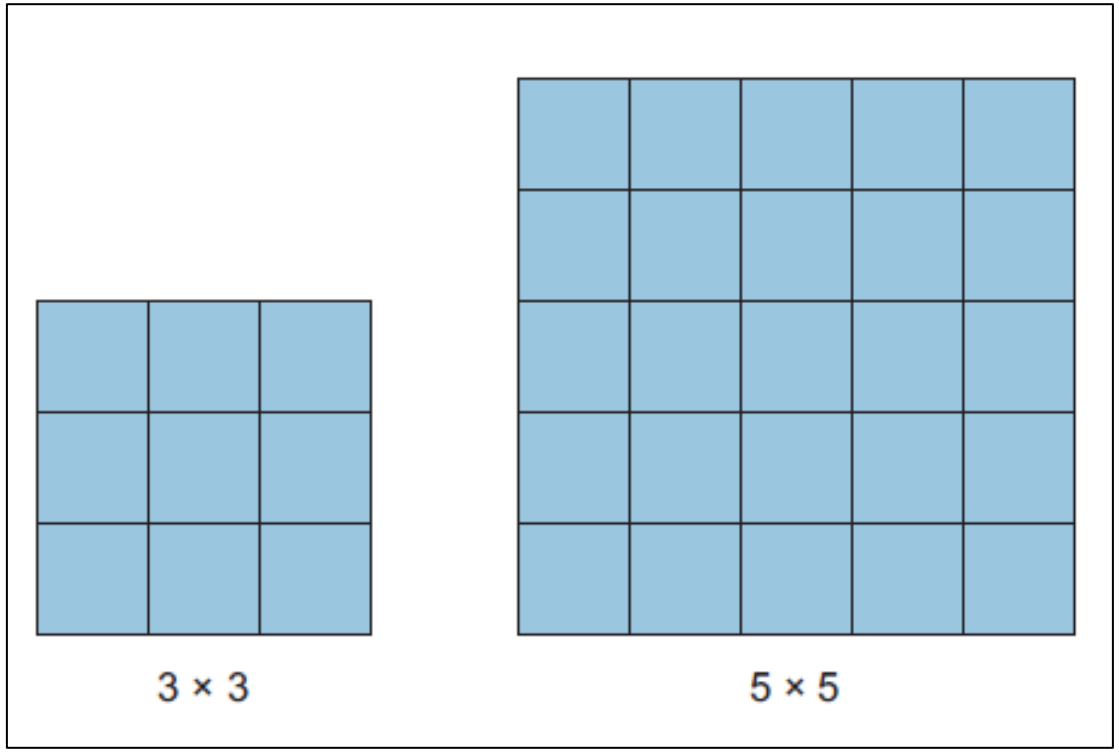
The deeper your network is (the more layers), the more it will learn the features of the dataset



Elgendy (2020)

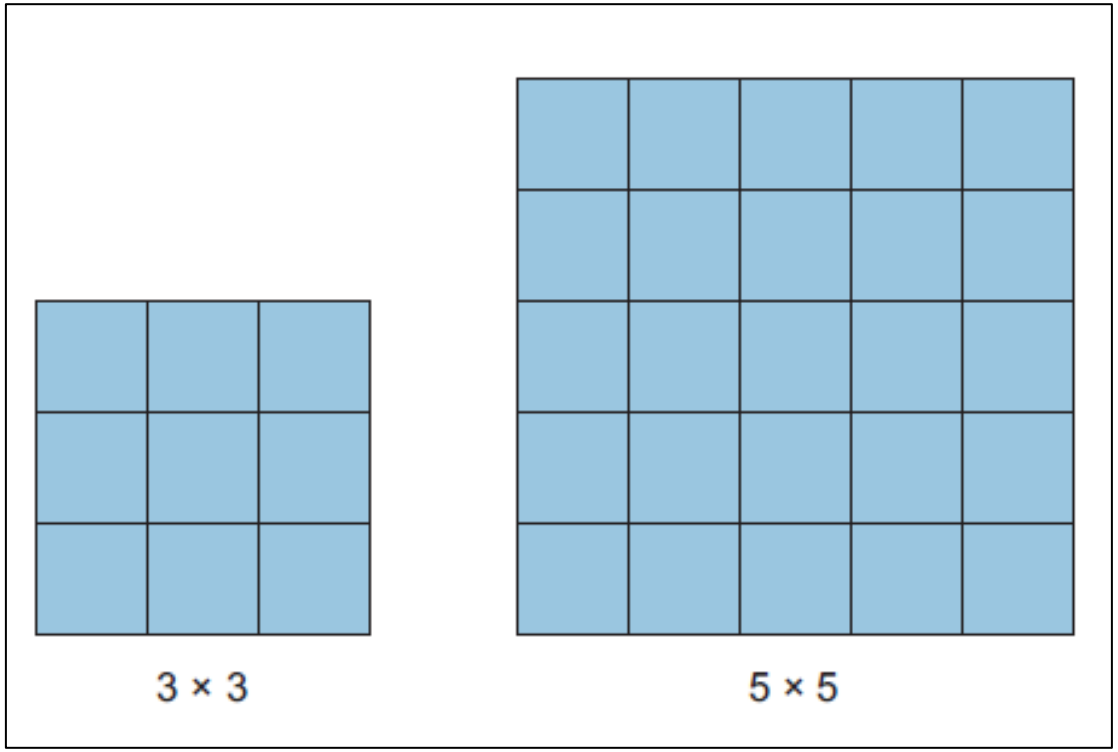
Kernel Size

- *A kernel* is a matrix of weights that slides over the image to extract features.
- **The kernel size** refers to the dimensions of the convolution filter (width times height).
- The intuition is that smaller filters will capture very fine details of the image, and bigger filters will miss minute details in the image.



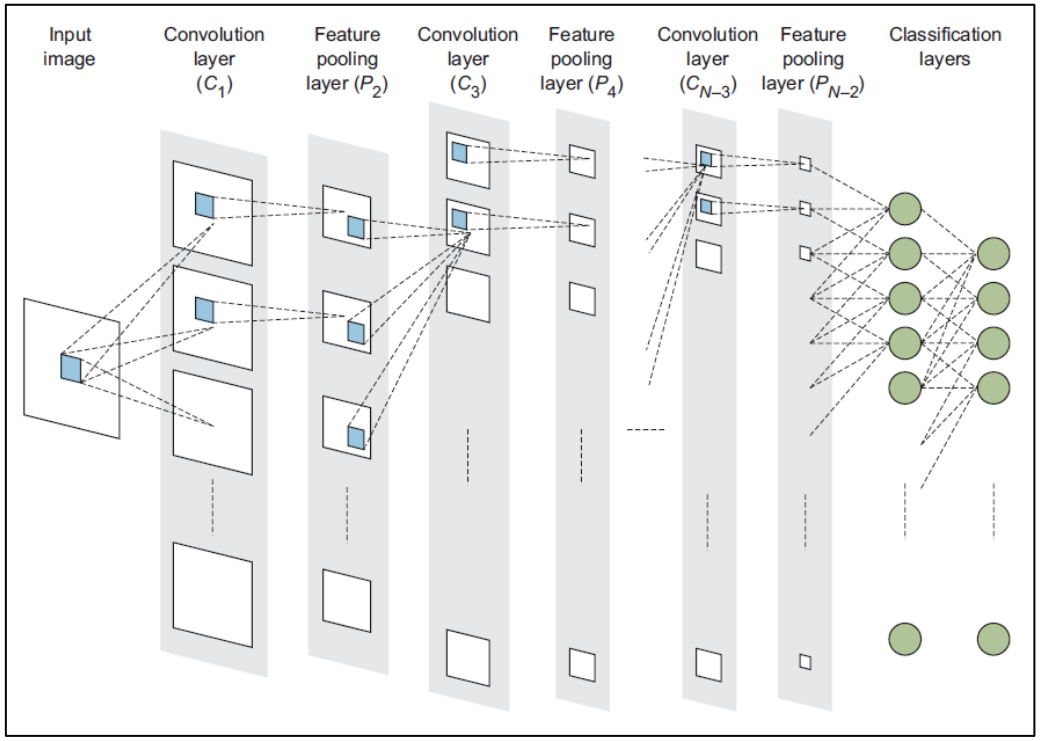
Kernel Size (2)

- Kernel filters are almost always square and range from the smallest at 2 × 2.
- Filters contain the weights that will be learned by the network.
  - So, the bigger the kernel_size, the higher the computational complexity.



Number Of Filters In The Convolutional Layer

- With CNNs, the convolutional layers are the hidden layers.
  - And to increase the number of **neurons in hidden layers**, we increase the **number of kernels in convolutional layers**.
  - Each kernel unit is considered a neuron.
- For example:
  - If we have a 3 × 3 kernel in the convolutional layer, this means we have 9 hidden units in this layer.
  - When we add another 3 × 3 kernel, we have 18 hidden units.
  - Add another one, and we have 27, and so on.
- So, by increasing the number of kernels in a convolutional layer, we increase the number of hidden units, which makes our network more complex and able to detect more complex patterns.



Strides And Padding

They both control the shape of the output of a convolutional layer.

- *Strides*—The amount by which the filter slides over the image.
  - For example, to slide the convolution filter one pixel at a time, the strides value is 1.
    - Strides of 1 will make the output image roughly the same height and width of the input image.
  - If we want to jump two pixels at a time, the strides value is 2.
    - Jumping pixels produces smaller output volumes spatially.
    - Strides of 2 will make the output image roughly half of the input image size.
  - Strides of 3 or more are uncommon and rare in practice.

Strides And Padding

They both control the shape of the output of a convolutional layer.

- *Padding*—Often called *zero-padding* because we add zeros around the border of an image.
  - Padding is most commonly used to allow us to preserve the spatial size of the input volume so the input and output width and height are the same.
    - This is important for building deeper networks, since, otherwise, the height/width would shrink as we went to deeper layers.

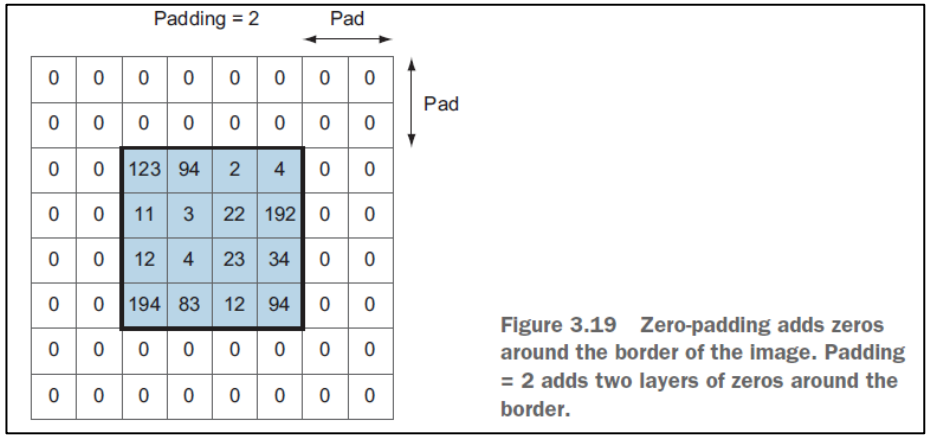
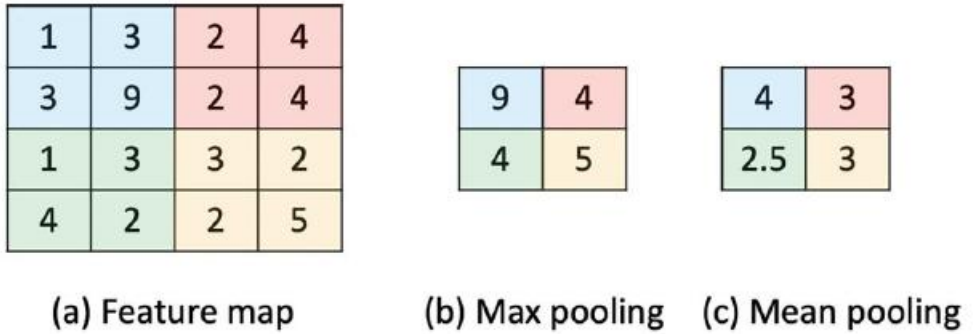


Figure 3.19 Zero-padding adds zeros around the border of the image. Padding = 2 adds two layers of zeros around the border.

Pooling Layer

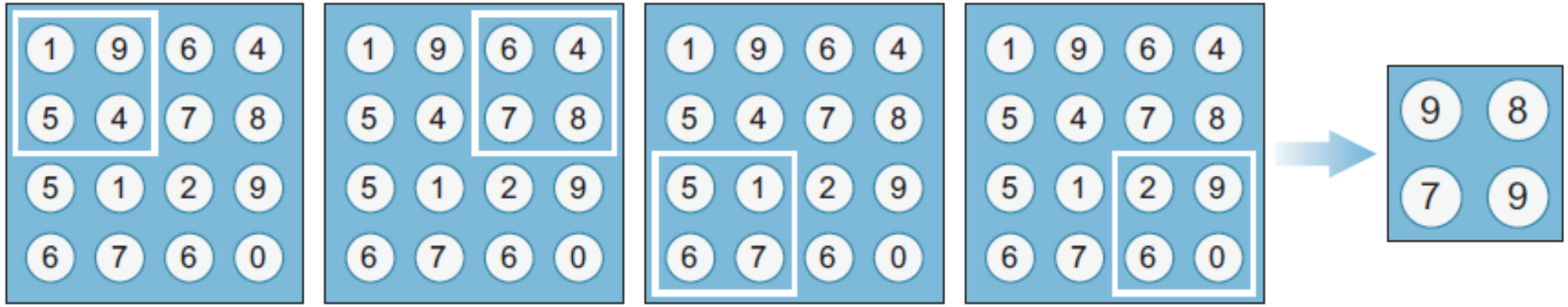


- Adding several convolutional layers (usually tens or even hundreds) will produce a huge number of parameters (weights).
  - This increase in network dimensionality increases the time and space complexity of the mathematical operations that take place in the learning process.
- *Pooling* helps reduce the size of the network by **reducing the number of parameters** passed to the next layer.
  - By applying a summary statistical function, such as a maximum or average.
  - Pooling layers are commonly added after every one or two convolutional layers.

Elgendy (2020); Han (2024)

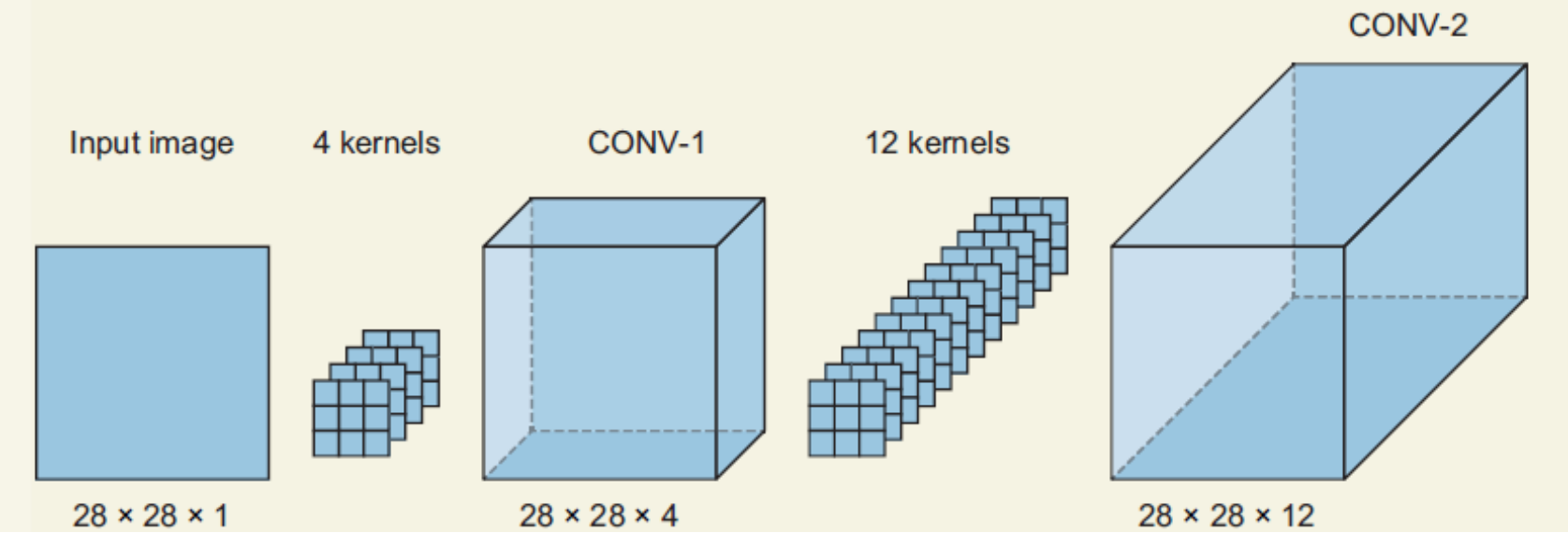
Pooling Layer: Max Pooling

- Similar to convolutional kernels, max pooling kernels are **windows** of a certain size and strides value that **slide** over the image.
  - The difference with max pooling is that the windows **don't have weights or any values**.
- All they do is slide over the feature map created by the previous convolutional layer and select the max pixel value to pass along to the next layer, ignoring the remaining values.
- For example: a pooling filter with a size of 2 × 2 and strides of 2 (the filter jumps 2 pixels when sliding over the image).
  - This pooling layer reduces the feature map size from 4 × 4 down to 2 × 2.



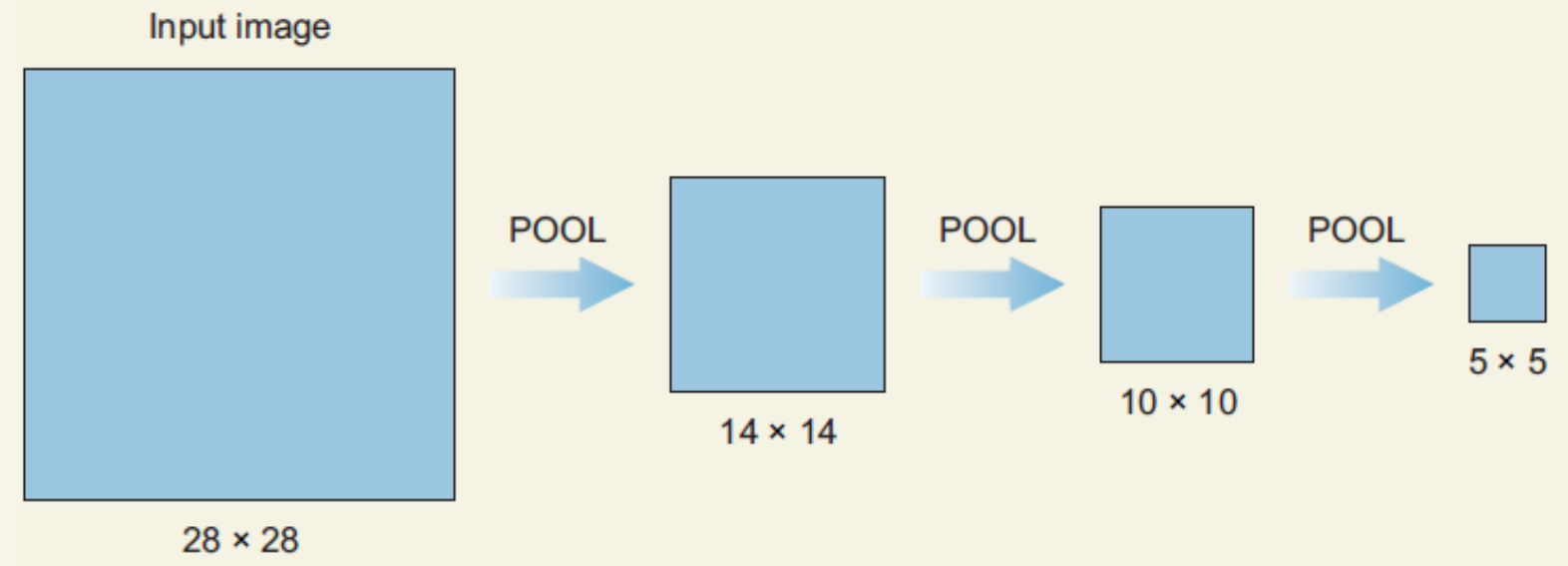
Visualize what happens after each layer

- After the convolutional layers, the image keeps its width and height dimensions (usually), but it gets deeper and deeper after each layer.
  - It is because of the cutting-the-image-into-pieces-of-features process.



Visualize what happens after each layer (2)

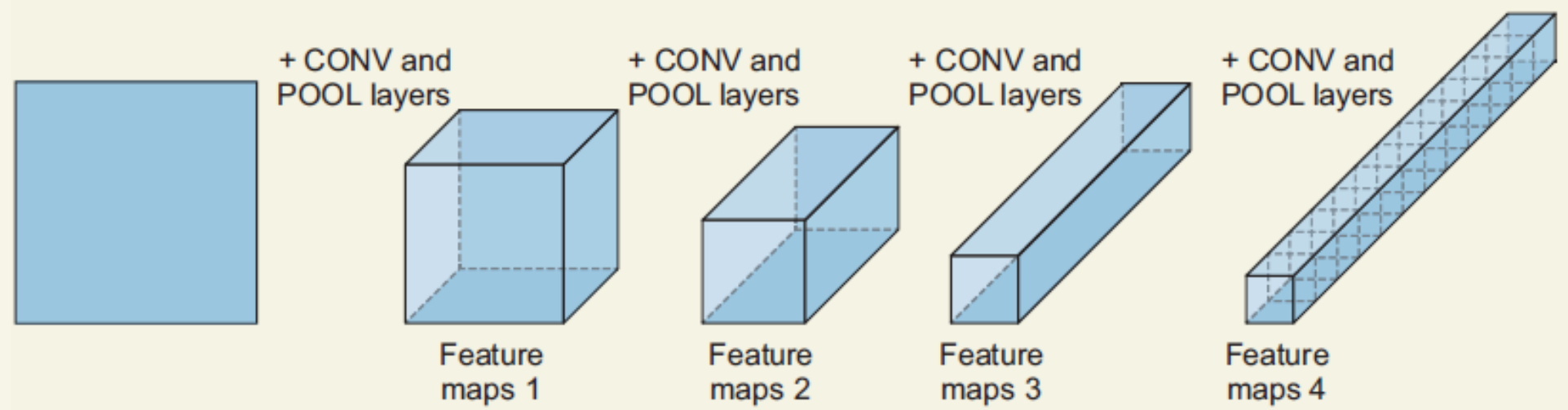
- After the pooling layers, the image keeps its depth but shrinks in width and height.



Elgendy (2020)

Visualize what happens after each layer (3)

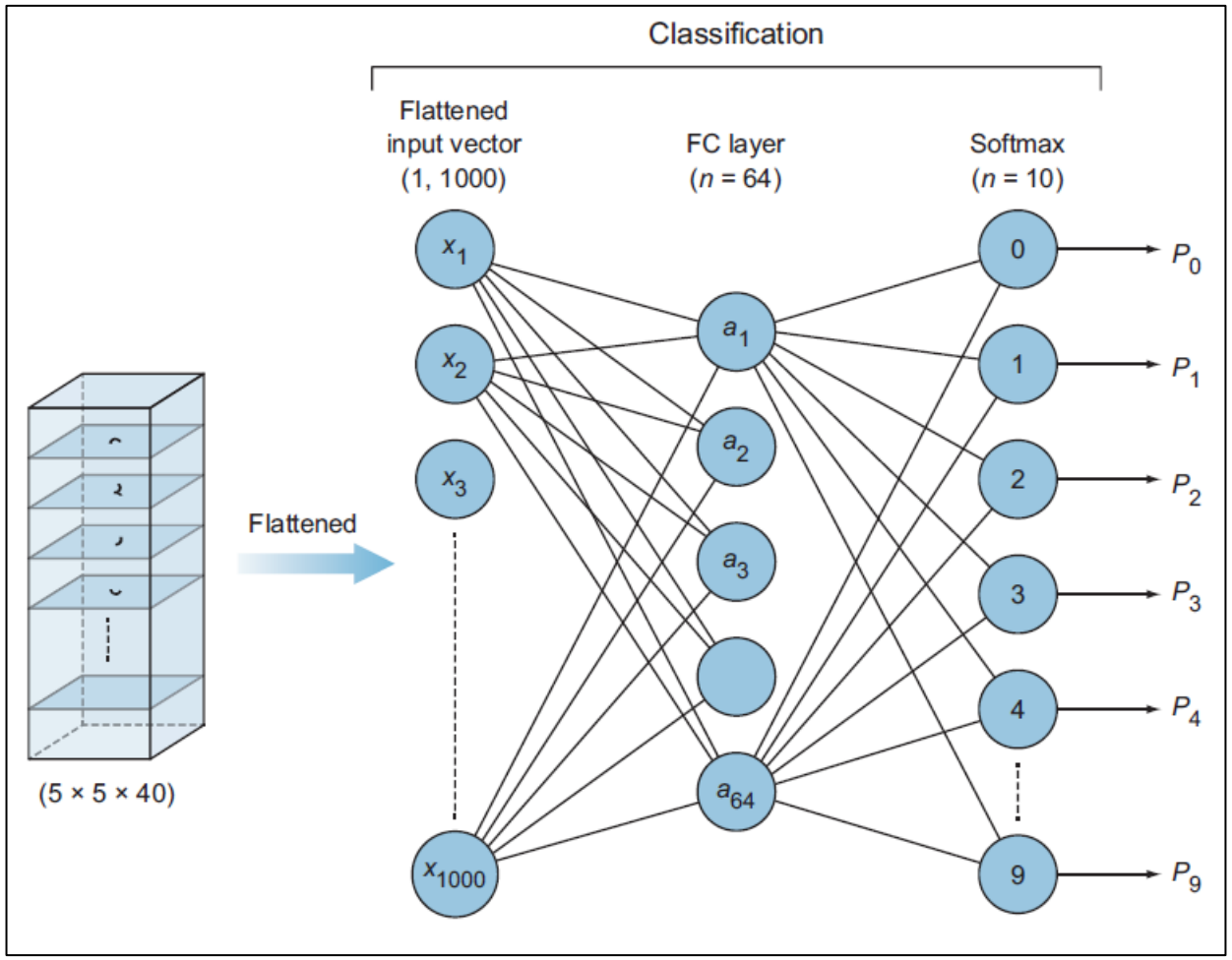
- Putting the convolutional and pooling together, we get **a long tube of small shaped images** that contain all the features in the original image.
  - 40 here as an example for the depth of the feature tube, as in 40 feature maps.



- The last step is to **flatten** this tube before feeding it to the **fully connected layer** for classification.
  - The flattened layer will have the dimensions of (1, m) where m = 5 × 5 × 40 = 1,000 neurons.

Fully Connected Layers for Classification

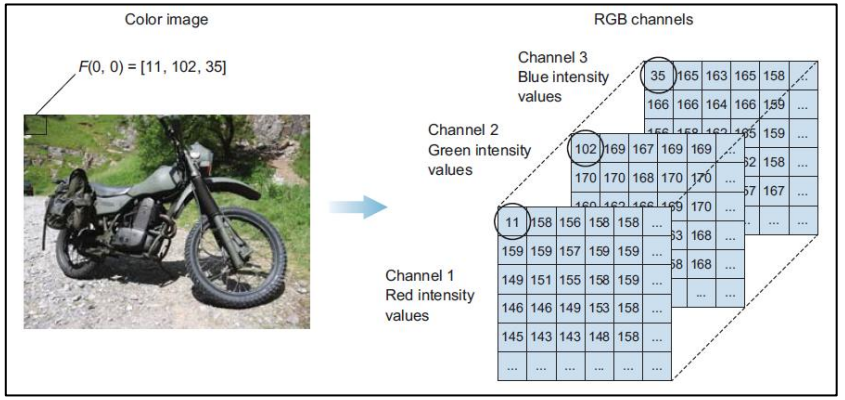
- *Input flattened vector*
  - To feed the features tube to the MLP for classification, we flatten it to a vector with the dimensions (1, *n*).
  - For example, if the features tube has the dimensions of 5 × 5 × 40, the flattened vector will be (1, 1000).
- *Hidden layer*
  - We add one or more fully connected layers.
- *Output layer*
  - The softmax activation function is recommended for classification problems involving more than two classes.
    - In this example, we are classifying digits from 0 to 9: 10 classes.
  - The number of neurons in the output layer is equal to the number of classes.
    - Thus, the output layer will have 10 nodes.
    - Each node represents the probability of each class we have.

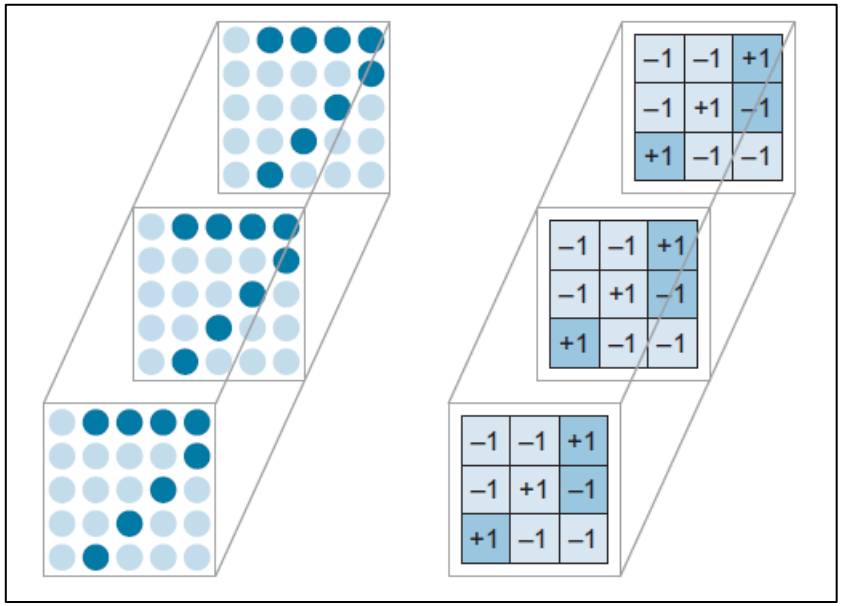


**The terms "MLP", "fully connected", "dense", and sometimes "feedforward" are used interchangeably to refer to the regular neural network architecture.*

Convolution over color images (3D images)?

- Color images are represented by three matrices.
- Now the kernel is itself three-dimensional: one dimension for each color channel.
- To perform convolution, we will do the same thing we did before, except that now, our sum is **three times** as many terms.
  - The **network complexity increases** because it has to optimize **more parameters**;
  - Color images also take up **more memory space**.





CNN Variants

- LeNet (1998): 3 convolutional layers and 2 fully connected layers.
- AlexNet (2012): 5 convolutional layers and 3 fully connected layers.
- VGGNet (2014): 13 convolutional layers and 3 fully connected layers. It also solved the problem of setting up the hyperparameters (kernel size, padding, strides, and so on) of the convolutional and pooling layers by creating a uniform configuration for them to be used across the entire network.
- Inception (2014): It uses a network 22 layers deep (deeper than VGGNet). It also tried to solve the same problem as VGGNet: instead of having to decide which filter size to use and where to add the pooling layer, Inception says, “Let’s use them all.”
- ResNet (2015)

CNN Variants: ResNet

- The Residual Neural Network (ResNet) (2015) attempted to solve the **vanishing gradient problem** when training **very deep** neural networks.
- The ResNet team introduced skip connections that allow information to flow from earlier layers in the network to later layers, creating an alternate **shortcut path** for the gradient to flow through.
  - This combination of the skip connection and convolutional layers is called a ***residual block***.
- The fundamental breakthrough with ResNet was that it allowed us to train extremely deep neural networks with **hundreds** of layers.

